
ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: LẬP TRÌNH THIẾT BỊ DI ĐỘNG

Đề tài: Ứng dụng nghe nhạc đa nền tảng sử dụng Flutter

Nhóm 06

Họ và tên:

Lê Thị Hương Giang - 23010140

Trần Quốc Huy - 23010184

Giảng viên hướng dẫn: ThS. Nguyễn Xuân Quế

Lớp học phần: CSE702027-1-2-25(N03)

Hà Nội – Năm 2026

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên	MSSV	Nội dung công việc	Tỉ lệ đóng góp	Nhóm chấm điểm	Giảng viên chấm điểm
Lê Thị Hương Giang	23010140	- Xây dựng các tệp: <code>app_localization.dart</code>, <code>user.dart</code>, <code>settings.dart</code>. - Thiết kế giao diện: <code>home_screen</code> (Tabs & Search), <code>now_playing_screen</code>. - Phối hợp xây dựng nội dung báo cáo và hệ thống biểu đồ UML	50%	10	
Trần Quốc Huy	23010184	- Lập trình toàn bộ các phần code còn lại của dự án (Logic phát nhạc, xử lý dữ liệu, Repository...). - Chịu trách nhiệm kiểm tra, sửa lỗi (Fix bugs) và tối ưu hóa mã nguồn. - Phối hợp xây dựng nội dung báo cáo và hệ thống biểu đồ UML.	50%	10	

LỜI CẢM ƠN

Trong quá trình thực hiện và hoàn thành báo cáo cũng như dự án “Xây dựng ứng dụng nghe nhạc đa nền tảng sử dụng Flutter”, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ, hướng dẫn và động viên từ phía giảng viên và bạn bè. Nhờ đó, nhóm có thể hoàn thành đề tài trong phạm vi yêu cầu của học phần.

Trước hết, nhóm xin gửi lời cảm ơn chân thành đến thầy ThS. Nguyễn Xuân Quế - giảng viên phụ trách học phần Lập trình cho thiết bị di động, người đã tận tình hướng dẫn, cung cấp kiến thức nền tảng và định hướng cho chúng em trong suốt quá trình học tập cũng như thực hiện dự án. Những góp ý và sự hướng dẫn của thầy/cô đã giúp nhóm hiểu rõ hơn về quy trình phát triển một ứng dụng di động và cách áp dụng các công nghệ hiện đại vào thực tế.

Bên cạnh đó, nhóm cũng xin cảm ơn các bạn trong lớp đã chia sẻ tài liệu, trao đổi kiến thức và hỗ trợ nhau trong quá trình học tập. Những ý kiến đóng góp và sự hợp tác của các bạn đã giúp nhóm có thêm nhiều kinh nghiệm và góc nhìn trong quá trình hoàn thiện dự án.

Nhóm cũng xin cảm ơn các thành viên trong nhóm đã cùng nhau thảo luận, phân chia công việc và phối hợp thực hiện dự án. Sự nỗ lực và tinh thần làm việc nhóm của các thành viên là yếu tố quan trọng giúp dự án được hoàn thành đúng tiến độ.

Mặc dù nhóm đã cố gắng hoàn thành báo cáo và ứng dụng trong khả năng của mình, tuy nhiên do kiến thức và kinh nghiệm thực tế còn hạn chế nên chắc chắn báo cáo vẫn còn những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp và nhận xét từ giảng viên để có thể hoàn thiện hơn trong các nghiên cứu và dự án tiếp theo.

Nhóm xin chân thành cảm ơn!

LỜI MỞ ĐẦU

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ di động đã làm thay đổi đáng kể cách con người tiếp cận và sử dụng các dịch vụ số. Các ứng dụng trên thiết bị di động ngày càng đóng vai trò quan trọng trong đời sống hằng ngày, từ học tập, làm việc cho đến giải trí. Trong đó, các ứng dụng nghe nhạc trực tuyến và ngoại tuyến đã trở thành một phần không thể thiếu đối với người dùng, giúp họ dễ dàng thưởng thức âm nhạc mọi lúc, mọi nơi thông qua các thiết bị thông minh.

Song song với sự phát triển của các nền tảng di động, các công nghệ phát triển ứng dụng đa nền tảng cũng ngày càng được quan tâm. Flutter là một trong những framework hiện đại do Google phát triển, cho phép lập trình viên xây dựng ứng dụng cho nhiều nền tảng khác nhau chỉ với một mã nguồn duy nhất. Với ưu điểm về hiệu năng, khả năng xây dựng giao diện linh hoạt và hệ sinh thái thư viện phong phú, Flutter đang trở thành một công cụ phổ biến trong việc phát triển ứng dụng di động hiện nay.

Xuất phát từ những xu hướng đó, nhóm chúng em thực hiện đề tài “Xây dựng ứng dụng nghe nhạc đa nền tảng sử dụng Flutter” trong khuôn khổ học phần Lập trình cho thiết bị di động. Mục tiêu của đề tài là xây dựng một ứng dụng nghe nhạc cơ bản với giao diện thân thiện, hỗ trợ phát nhạc và quản lý danh sách bài hát, đồng thời áp dụng các kỹ thuật lập trình Flutter để tổ chức cấu trúc dự án rõ ràng và dễ mở rộng.

Trong quá trình thực hiện dự án, nhóm đã sử dụng Flutter SDK kết hợp với ngôn ngữ lập trình Dart để xây dựng toàn bộ ứng dụng. Bên cạnh đó, một số thư viện hỗ trợ như `just_audio` được sử dụng để xử lý chức năng phát nhạc, và `flutter_localizations` được áp dụng để triển khai tính năng đa ngôn ngữ cho ứng dụng. Hệ thống được thiết kế theo cấu trúc module bao gồm các thành phần như giao diện người dùng, mô hình dữ liệu và lớp quản lý dữ liệu, giúp đảm bảo tính rõ ràng và khả năng mở rộng của dự án.

Thông qua quá trình thực hiện đề tài này, nhóm không chỉ củng cố kiến thức về lập trình Flutter mà còn hiểu rõ hơn về quy trình xây dựng một ứng dụng di động hoàn chỉnh, từ khâu phân tích yêu cầu, thiết kế hệ thống cho đến triển khai và kiểm thử ứng dụng. Đồng thời, dự án cũng giúp nhóm rèn luyện kỹ năng làm việc nhóm, phân chia nhiệm vụ và phối hợp trong quá trình phát triển phần mềm.

Mặc dù nhóm đã cố gắng hoàn thiện dự án trong phạm vi kiến thức và thời gian cho phép, nhưng do kinh nghiệm còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp và nhận xét từ giảng viên để có thể cải thiện và hoàn thiện hơn trong các dự án sau.

Nhóm chúng em xin chân thành cảm ơn thầy ThS. Nguyễn Xuân Quế đã hướng dẫn và hỗ trợ trong suốt quá trình thực hiện đề tài.

MỤC LỤC

BẢNG PHÂN CÔNG CÔNG VIỆC	1
LỜI CẢM ƠN	2
LỜI MỞ ĐẦU	3
MỤC LỤC	4
CHƯƠNG 1: GIỚI THIỆU	10
1.1 Giới thiệu đề tài	10
1.2 Các giải pháp đã có và giải pháp đề xuất	10
1.3 Mục tiêu của đề tài	10
1.4 Công nghệ sử dụng	10
CHƯƠNG 2: SRS – SOFTWARE REQUIREMENT SPECIFICATION	13
2.1 Mô tả hệ thống	13
2.2 Đối tượng sử dụng hệ thống	13
2.3 Use Case Diagram	13
2.3.1. Nhóm chức năng Trình phát nhạc (Core Player)	14
2.3.2. Nhóm chức năng Khám phá (Discovery & Navigation)	14
2.3.3. Nhóm chức năng Hệ thống và Tài khoản (System & Account)	15
2.4 Đặc tả Use Case	15
2.4.1. Use Case: Play Music	15
2.4.2. Use Case: Change Language	16
2.4.3. Use Case: Khám phá Playlist và Album (Discovery)	16
2.4.4. Use Case: Phát nhạc từ Playlist và Album (Discovery)	17
2.4.5. Use Case: Xem thông tin tài khoản (Account)	18
2.4.6. Use Case: Xem thông tin nhóm phát triển (About Us)	19
2.4.7. Use Case: Tìm kiếm bài hát	19
2.5 Yêu cầu chức năng	20
2.5.1 Nhóm chức năng Quản lý nội dung (Discovery & Home)	20
2.5.2 Nhóm chức năng Trình phát nhạc (Core Player)	20
2.5.3 Nhóm chức năng Hệ thống và Cá nhân hóa	21
2.6 Yêu cầu phi chức năng	21
2.6.1 Tính khả dụng và Giao diện (Usability & UI/UX)	21
2.6.2 Hiệu năng (Performance)	21
2.6.3 Tính ổn định và Tin cậy (Reliability)	21
2.6.4 Khả năng bảo trì và Mở rộng (Maintainability & Scalability)	22
2.6.5 Hỗ trợ đa ngôn ngữ (Localization)	22
CHƯƠNG 3: SAD – SOFTWARE ARCHITECTURE DESIGN	22
3.1 Kiến trúc hệ thống	22
3.2 Cấu trúc thư mục dự án	22
3.3 Thiết kế dữ liệu	23
3.3.1 Class Song	23
3.3.2 Class Source (Nguồn dữ liệu)	24
3.3.3 Class Repository	25
3.3.4 Class AudioManager (Service)	26
3.3.5 UI Screens (Presentation Layer)	26
3.4 Class Diagram	26
3.5 Activity Diagram	28

3.6 Sequence Diagram	34
3.7 Thiết kế giao diện	36
CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	46
4.1 Môi trường cài đặt và thử nghiệm	46
4.2 Kịch bản kiểm thử (Test Cases)	46
4.3 Phân tích và Xử lý tình huống (Case Studies)	46
4.3.1 Xử lý luồng dữ liệu khi tìm kiếm	46
4.3.2 Xử lý chuyển đổi ngôn ngữ thời gian thực (Real-time Localization)	47
4.3.3 Quản lý hàng chờ khi phát theo Playlist/Album	47
4.4 Đánh giá và Tổng kết thử nghiệm	47
CHƯƠNG 5: KẾT LUẬN	48
5.1 Kết luận	48
5.1.1 Những kết quả đạt được	48
5.1.2 Hạn chế của đề tài	48
5.2 Hướng phát triển trong tương lai	48
TÀI LIỆU THAM KHẢO	49

DANH MỤC HÌNH ẢNH

STT	Tên hình ảnh	Trang
1	Sơ đồ 1: Sơ đồ Use Case tổng quan hệ thống	12
2	Hình 1: đoạn mã Class Song	22
3	Hình 2: đoạn mã Class Source	23
4	Hình 3: đoạn mã Class Repository	24
5	Sơ đồ 2: Sơ đồ lớp của hệ thống	25
6	Sơ đồ 3: Sơ đồ hoạt động chức năng Phát nhạc	27
7	Sơ đồ 4: Sơ đồ hoạt động: Khám phá playlist và album	28
8	Sơ đồ 5: Sơ đồ hoạt động: Phát nhạc từ Playlist/Album	29
9	Sơ đồ 6: Sơ đồ hoạt động: Thay đổi ngôn ngữ	30
10	Sơ đồ 7: Sơ đồ hoạt động: Xem thông tin Tài khoản và Nhóm	31
11	Sơ đồ 8: Sơ đồ hoạt động: Tìm kiếm bài hát	32
12	Sơ đồ 9: Sơ đồ hoạt động: Luồng tổng quát của ứng dụng	33
13	Sơ đồ 10: Sơ đồ tuần tự: Chức năng phát nhạc	33
14	Sơ đồ 11: Sơ đồ tuần tự: Thay đổi ngôn ngữ	33
15	Sơ đồ 12: Sơ đồ tuần tự: Khám phá Playlist và Album	34
16	Sơ đồ 13: Sơ đồ tuần tự: Phát nhạc từ Playlist và Album	34
17	Sơ đồ 14: Sơ đồ tuần tự: Xem thông tin tài khoản	34
18	Sơ đồ 15: Sơ đồ tuần tự: Xem thông tin nhóm phát triển	35

19	Sơ đồ 16: Sơ đồ tuần tự: Tìm kiếm bài hát	35
20	Hình 4: Giao diện Home	36
21	Hình 5: Giao diện Khám phá	37
22	Hình 6: Danh sách Album	38
23	Hình 7: Danh sách bài hát trong Album	39
24	Hình 8: Giao diện Now Playing	40
25	Hình 9: Giao diện Settings	41
26	Hình 10: Giao diện Account	42
27	Hình 11: Giao diện Xem danh sách thông tin nhóm	43
28	Hình 12: Giao diện tìm kiếm bài hát	44
29	Hình A: Framework Flutter	9
30	Hình B: Ngôn ngữ lập trình Dart	9

DANH MỤC BẢNG BIỂU

STT	Tên bảng	Trang
1	Bảng 1: Đặc tả Use Case 01 - Play Music	13
2	Bảng 2: Đặc tả Use Case 02 - Change Language	14
3	Bảng 3: Đặc tả Use Case 03 - Discovery	14
4	Bảng 4: Đặc tả Use Case 04 - Phát nhạc từ Playlist/Album	15
5	Bảng 5: Đặc tả Use Case 05 - Xem thông tin tài khoản	16
6	Bảng 6: Đặc tả Use Case 06 - Xem thông tin nhóm thực hiện dự án	17
7	Bảng 7: Đặc tả Use Case 07 - Tìm kiếm bài hát	18
8	Bảng 8. Kịch bản kiểm thử	45

Các chữ Tiếng Anh / Viết tắt

STT	Chữ Tiếng Anh / Viết Tắt	Giải thích
1	Music Player	Phát nhạc
2	UI	User Interface - Giao diện người dùng
3	Now_Playing	Đang phát nhạc
4	Play	Phát
5	Pause	Dừng
6	Next	Tiếp theo
7	Previous	Trước đó
8	User	Người dùng
9	Change language	Thay đổi ngôn ngữ
10	Setting	Cài đặt
11	Account	Tài khoản
12	UX	User Experience - Trải nghiệm người dùng
13	Data	Dữ liệu
14	Business Logic	Suy nghĩ nghiệp vụ
15	Layer	Lớp
16	Activity Diagram	Sơ đồ hoạt động
17	Sequence Diagram	Sơ đồ tuần tự

CHƯƠNG 1: GIỚI THIỆU

1.1 Giới thiệu đề tài

Trong thời đại công nghệ hiện nay, các ứng dụng giải trí trên thiết bị di động ngày càng phát triển mạnh mẽ. Trong đó, ứng dụng nghe nhạc là một trong những loại ứng dụng phổ biến nhất, giúp người dùng thưởng thức âm nhạc mọi lúc mọi nơi.

Nhằm mục đích học tập và thực hành phát triển ứng dụng di động, nhóm đã xây dựng ứng dụng **Music Player** sử dụng framework Flutter. Ứng dụng cho phép người dùng phát nhạc, chuyển bài, quản lý danh sách bài hát và hỗ trợ đa ngôn ngữ.

1.2 Các giải pháp đã có và giải pháp đề xuất

- Các giải pháp hiện có: Đã có nhiều ứng dụng nghe nhạc nổi tiếng trên thị trường, nhưng thường gặp phải các vấn đề như quảng cáo gây gián đoạn, yêu cầu trả phí cao, hoặc tiêu tốn quá nhiều tài nguyên hệ thống cho những tính năng không cần thiết.
- Giải pháp đề xuất : Nhóm đề xuất xây dựng ứng dụng bằng framework Flutter. Giải pháp này không chỉ giúp tối ưu hóa thời gian phát triển (viết code một lần, chạy trên nhiều nền tảng) mà còn mang lại giao diện mượt mà, hỗ trợ tốt khả năng đa ngôn ngữ để tiếp cận nhiều đối tượng người dùng.

1.3 Mục tiêu của đề tài

Dự án được thực hiện với các mục tiêu cụ thể sau:

- **Xây dựng ứng dụng di động:** Sử dụng thành thạo framework Flutter và ngôn ngữ Dart để phát triển sản phẩm hoàn chỉnh.
- **Thiết kế trải nghiệm người dùng:** Tạo ra giao diện trực quan, thân thiện và dễ sử dụng cho mọi đối tượng.
- **Tích hợp trình phát nhạc:** Triển khai thành công bộ phát nhạc với đầy đủ tính năng điều khiển cơ bản.
- **Hỗ trợ đa ngôn ngữ:** Tích hợp cả hai ngôn ngữ English và Vietnamese để tăng tính linh hoạt.
- **Quản lý mã nguồn:** Thiết kế cấu trúc dự án rõ ràng, khoa học, giúp dễ dàng bảo trì và phát triển thêm tính năng trong tương lai, sử dụng git để quản lí mã nguồn.

1.4 Công nghệ sử dụng

Để xây dựng ứng dụng **Music Player Demo**, nhóm đã sử dụng các công nghệ và thư viện phù hợp với việc phát triển ứng dụng di động đa nền tảng. Các công nghệ này giúp đảm bảo ứng dụng có hiệu năng tốt, dễ mở rộng và hỗ trợ nhiều hệ điều hành.

1.4.1 Flutter SDK



Hình A. Framework Flutter

Flutter là framework mã nguồn mở do Google phát triển, cho phép xây dựng ứng dụng đa nền tảng từ một codebase duy nhất. Flutter sử dụng kiến trúc dựa trên widget để xây dựng giao diện người dùng linh hoạt và có khả năng tùy biến cao.

Trong dự án này, Flutter được sử dụng để:

- Xây dựng toàn bộ **giao diện người dùng (UI)** của ứng dụng.
- Quản lý **navigation giữa các màn hình**.
- Tổ chức cấu trúc dự án theo các module như `ui`, `data`, `repository`, `model`.
- Hỗ trợ biên dịch ứng dụng sang nhiều nền tảng như:
 - Android
 - iOS
 - Web
 - Windows
 - macOS
 - Linux

Việc sử dụng Flutter giúp giảm đáng kể thời gian phát triển và đảm bảo tính nhất quán của giao diện trên các nền tảng khác nhau.

1.4.2 Ngôn ngữ lập trình Dart



Hình B. Ngôn ngữ lập trình Dart

Dart là ngôn ngữ lập trình do Google phát triển và là ngôn ngữ chính được sử dụng trong Flutter. Dart hỗ trợ cả lập trình hướng đối tượng và lập trình bất đồng bộ, rất phù hợp cho việc xây dựng ứng dụng giao diện người dùng hiện đại.

Trong dự án này, Dart được sử dụng để:

- Xây dựng logic của ứng dụng.
- Định nghĩa các **model dữ liệu**, ví dụ như `Song`.
- Xây dựng các lớp quản lý dữ liệu trong thư mục `repository`.
- Xây dựng các widget giao diện trong thư mục `ui`.

- Quản lý trạng thái và điều khiển các chức năng của trình phát nhạc.

Nhờ cú pháp rõ ràng và hiệu năng cao, Dart giúp việc phát triển ứng dụng Flutter trở nên hiệu quả và dễ bảo trì.

1.4.3 Thư viện `just_audio`

`just_audio` là một thư viện Flutter chuyên dụng để xử lý việc phát âm thanh. Thư viện này cung cấp các API đơn giản nhưng mạnh mẽ giúp ứng dụng có thể phát và điều khiển audio một cách ổn định.

Trong dự án, thư viện này được sử dụng để:

- Phát các tệp nhạc trong ứng dụng.
- Điều khiển trạng thái phát nhạc như:
 - Play
 - Pause
 - Next
 - Previous
- Quản lý tiến trình phát nhạc trong màn hình **Now Playing**.

Việc sử dụng `just_audio` giúp đảm bảo khả năng phát nhạc ổn định trên nhiều nền tảng khác nhau.

1.4.4 Thư viện `flutter_localizations` và hệ thống Localization

Để hỗ trợ người dùng sử dụng ứng dụng bằng nhiều ngôn ngữ khác nhau, dự án sử dụng hệ thống **internationalization (i18n)** của Flutter thông qua thư viện `flutter_localizations`.

Trong dự án, hệ thống localization được triển khai thông qua:

- Các file ngôn ngữ dạng `.arb`:
 - `app_en.arb`
 - `app_vi.arb`
- Các file được Flutter tự động sinh ra:
 - `app_localizations.dart`
 - `app_localizations_en.dart`
 - `app_localizations_vi.dart`

Chức năng của hệ thống này bao gồm:

- Cho phép ứng dụng hỗ trợ **hai ngôn ngữ: tiếng Anh và tiếng Việt**.
- Tự động hiển thị nội dung theo ngôn ngữ đã chọn.
- Dễ dàng mở rộng để hỗ trợ thêm các ngôn ngữ khác trong tương lai.

Nhờ đó, ứng dụng trở nên thân thiện hơn với nhiều đối tượng người dùng khác nhau.

1.4.5 Hệ thống quản lý dự án Flutter

Ngoài các công nghệ chính, dự án còn sử dụng hệ thống quản lý dự án mặc định của Flutter, bao gồm:

- `pubspec.yaml`: quản lý thư viện và tài nguyên của dự án.
- `assets`: lưu trữ các tài nguyên như hình ảnh hoặc tệp âm thanh.
- `analysis_options.yaml`: cấu hình quy tắc kiểm tra mã nguồn.
- `test`: chứa các file kiểm thử widget.

Những thành phần này giúp tổ chức dự án một cách khoa học và hỗ trợ quá trình phát triển, kiểm thử và bảo trì ứng dụng.

CHƯƠNG 2: SRS – SOFTWARE REQUIREMENT SPECIFICATION

2.1 Mô tả hệ thống

Ứng dụng Music Player là một giải pháp di động tập trung vào trải nghiệm nghe nhạc cá nhân. Hệ thống cho phép người dùng tương tác trực tiếp với danh sách các tệp âm thanh thông qua giao diện trực quan. Hiện tại, dữ liệu về bài hát như tên, nghệ sĩ và đường dẫn tệp được lưu trữ cục bộ và quản lý thông qua các Model trong thư mục `data/model` của dự án để đảm bảo tốc độ truy xuất nhanh.

2.2 Đối tượng sử dụng hệ thống

Hệ thống có một loại người dùng chính:

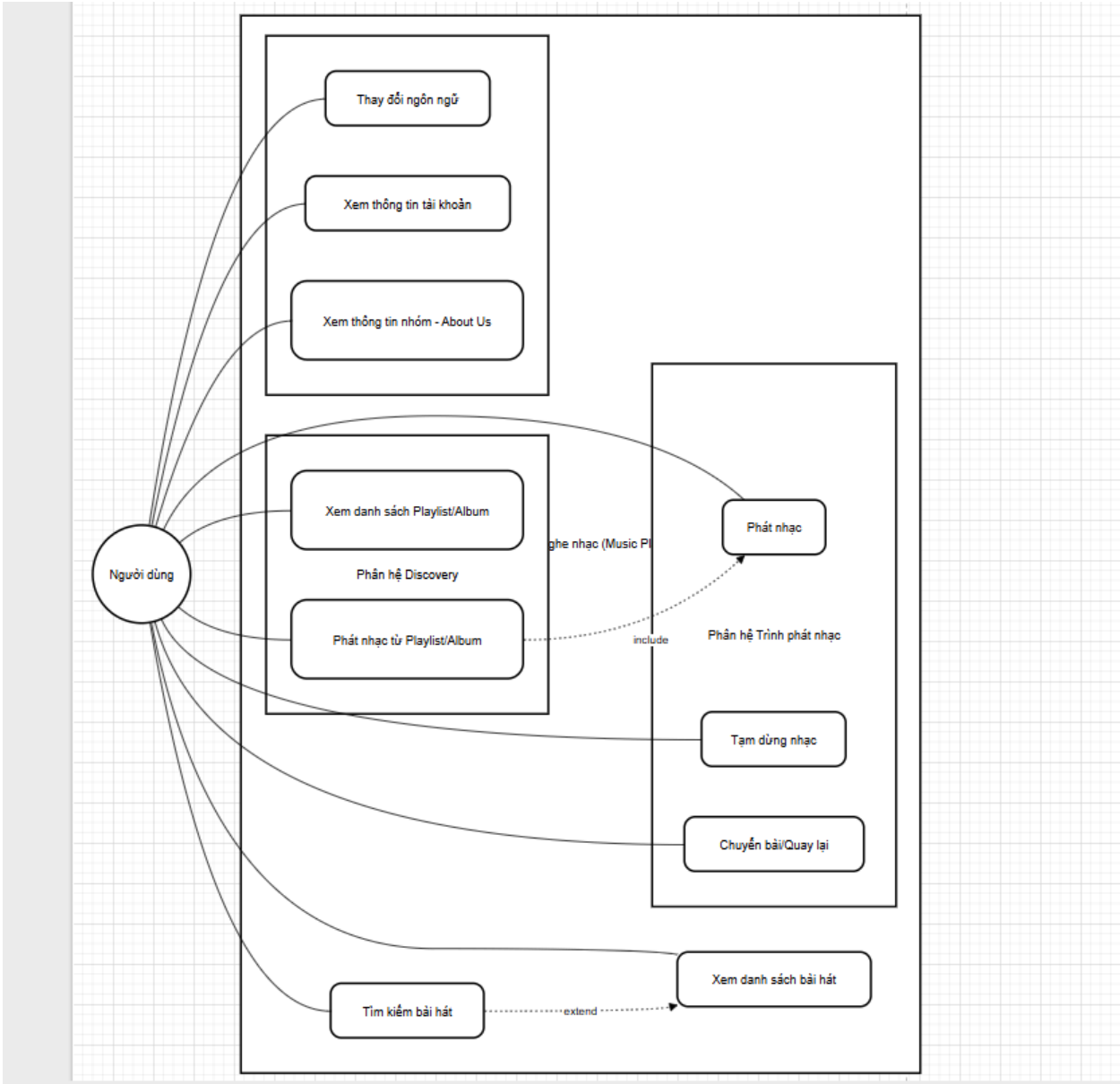
User (Người dùng)

Hệ thống được thiết kế dành cho một đối tượng người dùng duy nhất:

- **User (Người dùng):** Là người trực tiếp sử dụng ứng dụng để nghe nhạc và tùy chỉnh giao diện. Người dùng có các quyền hạn và khả năng tương tác sau:
- Xem toàn bộ danh sách bài hát có sẵn.
- Lựa chọn bài hát yêu thích từ danh sách.
- Điều khiển trình phát nhạc (Phát, Tạm dừng, Chuyển bài, Quay lại).
- Thay đổi ngôn ngữ hiển thị (Tiếng Anh hoặc Tiếng Việt).
- Xem thông tin giới thiệu về nhóm phát triển.

2.3 Use Case Diagram

Sơ đồ Use Case dưới đây mô tả các tương tác giữa Người dùng và các chức năng chính của ứng dụng:



Sơ đồ 1: Sơ đồ Use Case tổng quan hệ thống

Các Use Case trọng tâm bao gồm:

2.3.1. Nhóm chức năng Trình phát nhạc (Core Player)

Đây là nhóm chức năng quan trọng nhất, cho phép người dùng tương tác trực tiếp với nội dung âm nhạc:

- **Phát nhạc (Play Music):** Kích hoạt luồng âm thanh từ tệp dữ liệu đã chọn.
- **Tạm dừng nhạc (Pause Music):** Ngắt luồng âm thanh đang phát.
- **Chuyển bài tiếp theo (Next Song):** Chuyển sang bài hát kế tiếp trong danh sách.
- **Quay lại bài trước (Previous Song):** Phát lại bài hát vừa nghe hoặc bài đứng trước trong danh sách.

2.3.2. Nhóm chức năng Khám phá (Discovery & Navigation)

Giúp người dùng tìm kiếm và lựa chọn nội dung để nghe thông qua các giao diện chuyên biệt:

- **Xem danh sách bài hát (View Song List):** Hiển thị toàn bộ các bài hát có sẵn trong hệ thống.
- **Xem danh sách Playlist (View Playlists):** Khám phá các danh sách nhạc được phân loại theo chủ đề tại màn hình Discovery.

- **Xem danh sách Album (View Albums):** Duyệt các bài hát được gom nhóm theo Album của nghệ sĩ tại màn hình Discovery.

2.3.3. Nhóm chức năng Hệ thống và Tài khoản (System & Account)

Quản lý thông tin cá nhân và cấu hình ứng dụng:

- **Thay đổi ngôn ngữ (Change Language):** Chuyển đổi giao diện giữa tiếng Anh và tiếng Việt thông qua cài đặt.
- **Quản lý thông tin Tài khoản (Manage Account):** Xem và cập nhật thông tin người dùng tại màn hình User/Account.
- **Xem thông tin nhóm phát triển (View About Us):** Truy cập thông tin về các thành viên thực hiện dự án và mục tiêu ứng dụng.

2.4 Đặc tả Use Case

2.4.1. Use Case: Play Music

Thuộc tính	Mô tả
Use Case ID	UC01
Use Case Name	Play Music
Actor	User
Mục đích	Cho phép người dùng nghe bài hát đã chọn
Mô tả	Người dùng thực hiện thao tác phát bài hát từ danh sách
Điều kiện bắt buộc	Người dùng đã chọn một bài hát cụ thể
Điều kiện sau	Bài hát phát qua thiết bị đầu ra âm thanh
Mối quan hệ	Không
Kịch bản chính	1. User chọn bài hát từ danh sách 2. Hệ thống tải file nhạc từ bộ nhớ hoặc đường dẫn 3. Hệ thống bắt đầu phát nhạc 4. Hiển thị thông tin bài hát và trình điều khiển trên màn hình Now Playing
Kịch bản phụ	A1: Lỗi tải file nhạc 1. Hệ thống không thể tải file từ audioPath 2. Hệ thống hiển thị thông báo lỗi file không tồn tại 3. User chọn bài hát khác

Bảng 1: Đặc tả Use Case 01 - Play Music

2.4.2. Use Case: Change Language

Thuộc tính	Mô tả
Use Case ID	UC02
Use Case Name	Thay đổi ngôn ngữ
Actor	User
Mục đích	Thay đổi ngôn ngữ hiển thị của ứng dụng
Mô tả	Người dùng lựa chọn giữa Tiếng Anh hoặc Tiếng Việt
Điều kiện bắt buộc	Người dùng ở màn hình Settings
Điều kiện sau	Toàn bộ giao diện hiển thị theo ngôn ngữ mới đã chọn
Mối quan hệ	Không
Kịch bản chính	1. User truy cập vào trang Settings 2. Hệ thống hiển thị các tùy chọn ngôn ngữ (English/Vietnamese) 3. User chọn ngôn ngữ mong muốn 4. Hệ thống cập nhật các file <code>app_en.arb</code> hoặc <code>app_vi.arb</code> tương ứng 5. Giao diện thay đổi ngôn ngữ ngay lập tức
Kịch bản phụ	Không

Bảng 2: Đặc tả Use Case 02 - Change Language

2.4.3. Use Case: Khám phá Playlist và Album (Discovery)

Thuộc tính	Mô tả
Use Case ID	UC03
Use Case Name	Khám phá (Discovery)
Actor	User
Mục đích	Cho phép người dùng tìm kiếm nhạc theo danh mục Playlist và Album
Mô tả	Người dùng xem các gợi ý bài hát phổ biến và danh sách Album

Điều kiện bắt buộc	Ứng dụng đã khởi động thành công
Điều kiện sau	Danh sách nội dung gợi ý được hiển thị
Mối quan hệ	Không
Kịch bản chính	1. User truy cập vào màn hình Discovery 2. Hệ thống hiển thị danh sách các Playlist gợi ý 3. Hệ thống hiển thị danh sách các Album 4. User nhấn vào một Playlist/Album để xem chi tiết bài hát
Kịch bản phụ	A1: Không có dữ liệu gợi ý 1. Hệ thống kiểm tra nguồn dữ liệu từ repository 2. Nếu trống, hệ thống hiển thị thông báo "Chưa có nội dung gợi ý"

Bảng 3: Đặc tả Use Case 03 - Discovery

2.4.4. Use Case: Phát nhạc từ Playlist và Album (Discovery)

Thuộc tính	Mô tả
Use Case ID	UC04
Use Case Name	Phát nhạc từ Playlist/Album
Actor	User
Mục đích	Cho phép người dùng chọn và phát danh sách bài hát theo chủ đề và album
Mô tả	Người dùng truy cập danh mục, chọn Playlist/Album và bắt đầu phát nhạc
Điều kiện bắt buộc	Ứng dụng đã được khởi động và có dữ liệu trong Discovery
Điều kiện sau	Danh sách nhạc của Playlist/Album được tải và bài hát đầu tiên (hoặc bài hát được chọn) sẽ phát
Mối quan hệ	Bao gồm (Include) UC01: Phát nhạc
Kịch bản chính	1. User nhấn vào tab "Discovery" trên thanh điều hướng 2. Hệ thống hiển thị các danh mục Playlist và Album 3. User chọn một Playlist hoặc Album cụ thể 4. Hệ thống hiển thị danh sách các bài hát thuộc danh mục đó

	5. User chọn một bài hát trong danh sách để phát 6. Hệ thống tải toàn bộ danh sách vào hàng chờ (Queue) 7. Hệ thống chuyển hướng đến màn hình Now Playing và bắt đầu phát nhạc
Kịch bản phụ	A1: Danh mục trống 1. Hệ thống kiểm tra danh mục không có bài hát 2. Hệ thống hiển thị thông báo "Danh sách trống" 3. User quay lại chọn danh mục khác

Bảng 4: Đặc tả Use Case 04 - Phát nhạc từ Playlist/Album

2.4.5. Use Case: Xem thông tin tài khoản (Account)

Thuộc tính	Mô tả
Use Case ID	UC05
Use Case Name	Xem thông tin tài khoản
Actor	User
Mục đích	Cho phép người dùng xem thông tin cá nhân của họ trên ứng dụng
Mô tả	Người dùng truy cập vào phần tài khoản để kiểm tra các thông tin định danh cá nhân
Điều kiện bắt buộc	Người dùng đang ở màn hình
Điều kiện sau	Thông tin tài khoản được hiển thị đầy đủ trên màn hình
Mối quan hệ	Không
Kịch bản chính	1. User nhấn vào biểu tượng "Account" hoặc "User" trên thanh điều hướng 2. Hệ thống chuyển hướng đến màn hình User Screen 3. Hệ thống hiển thị các thông tin: Tên người dùng, Ảnh đại diện (nếu có) 4. User xem xong và có thể quay lại màn hình trước đó
Kịch bản phụ	Không

Bảng 5: Đặc tả Use Case 05 - Xem thông tin tài khoản

2.4.6. Use Case: Xem thông tin nhóm phát triển (About Us)

Thuộc tính	Mô tả
Use Case ID	UC06
Use Case Name	Xem thông tin nhóm phát triển
Actor	User
Mục đích	Cung cấp thông tin về người phát triển hoặc nhóm phát triển ứng dụng
Mô tả	Người dùng xem chi tiết về nhóm và các thành viên thực hiện dự án
Điều kiện bắt buộc	Người dùng truy cập vào mục “About Us” hoặc thông tin trong màn hình User
Điều kiện sau	Thông tin về nhóm 06 được hiển thị
Mối quan hệ	Không
Kịch bản chính	1. User chọn mục "About Us" hoặc xem trong phần thông tin tại Settings 2. Hệ thống hiển thị thông tin chi tiết: - Nhóm thực hiện: Nhóm 06 - Thành viên: Lê Thị Hương Giang & Trần Quốc Huy 3. User thoát khỏi màn hình thông tin
Kịch bản phụ	Không

Bảng 6: Đặc tả Use Case 06 - Xem thông tin nhóm thực hiện dự án

2.4.7. Use Case: Tìm kiếm bài hát

Thuộc tính	Mô tả
Use Case ID	UC07
Use Case Name	Tìm kiếm bài hát
Actor	User
Mục đích	Cho phép người dùng tìm kiếm nhanh bài hát theo tên bài hát hoặc tên tác giả.
Mô tả	Người dùng nhập từ khóa vào thanh tìm kiếm, hệ thống sẽ lọc và hiển thị danh sách các bài hát phù hợp.

Điều kiện bắt buộc	Người dùng đang ở màn hình có chức năng tìm kiếm (Home)
Điều kiện sau	Danh sách các bài hát khớp với từ khóa được hiển thị trên màn hình.
Mối quan hệ	Không
Kịch bản chính	1. User nhấn vào thanh tìm kiếm (Search Bar). 2. User nhập từ khóa (tên bài hát hoặc tên nghệ sĩ). 3. Hệ thống thực hiện lọc dữ liệu dựa trên từ khóa đã nhập. 4. Hệ thống hiển thị danh sách các bài hát có tên hoặc tên tác giả chứa từ khóa. 5. User chọn bài hát mong muốn từ kết quả tìm kiếm để phát hoặc xem chi tiết.
Kịch bản phụ	A1: Không tìm thấy kết quả 1. Hệ thống không tìm thấy bài hát nào khớp với từ khóa. 2. Hệ thống hiển thị thông báo: "Không tìm thấy bài hát phù hợp". A2: Từ khóa trống 1. User xóa từ khóa trong thanh tìm kiếm. 2. Hệ thống hiển thị lại danh sách bài hát mặc định ban đầu.

Bảng 7: Đặc tả Use Case 07 - Tìm kiếm bài hát

2.5 Yêu cầu chức năng

Yêu cầu chức năng mô tả các dịch vụ mà ứng dụng Music Player phải cung cấp, cách hệ thống phản ứng với các đầu vào cụ thể và cách hệ thống hoạt động trong các tình huống nhất định.

2.5.1 Nhóm chức năng Quản lý nội dung (Discovery & Home)

- **Hiển thị danh sách bài hát tổng quát:** Hệ thống phải liệt kê được toàn bộ các bài hát hiện có trong nguồn dữ liệu (Source) tại màn hình Home để người dùng dễ dàng lựa chọn.
- **Khám phá theo danh mục (Discovery):** * Hệ thống phải phân loại và hiển thị âm nhạc theo danh sách phát (Playlist) gợi ý.
 - Hệ thống phải hiển thị danh sách các Album nhạc để người dùng khám phá theo chủ đề hoặc nghệ sĩ.
- **Xem chi tiết thông tin bài hát:** Khi một bài hát được chọn, hệ thống phải hiển thị đầy đủ tiêu đề (Title), tên nghệ sĩ (Artist) và ảnh bìa (Image).

2.5.2 Nhóm chức năng Trình phát nhạc (Core Player)

Đây là nhóm chức năng xử lý nghiệp vụ chính của ứng dụng:

- **Phát nhạc (Play):** Cho phép người dùng bắt đầu nghe bài hát đã chọn từ danh sách.
- **Tạm dừng (Pause):** Cho phép người dùng tạm dừng bài hát đang phát và giữ nguyên vị trí thời gian của bài hát.
- **Chuyển bài tiếp theo (Next):** Tự động hoặc cho phép người dùng chuyển sang bài hát kế tiếp trong danh sách phát.
- **Quay lại bài trước (Previous):** Cho phép người dùng quay lại bài hát vừa phát trước đó.

2.5.3 Nhóm chức năng Hệ thống và Cá nhân hóa

- **Đa ngôn ngữ (Localization):** * Hệ thống phải cho phép người dùng chuyển đổi ngôn ngữ giao diện giữa Tiếng Anh và Tiếng Việt.
 - Toàn bộ các nhãn (labels), nút bấm và thông báo phải cập nhật ngay lập tức sau khi thay đổi ngôn ngữ.
- **Quản lý giao diện (Theme):** Hỗ trợ thay đổi giao diện sáng/tối (Light/Dark mode) theo cấu hình hệ thống hoặc tùy chọn của người dùng.
- **Thông tin tài khoản và định danh:** * Hiển thị thông tin định danh của người dùng tại màn hình Account.
 - Cung cấp thông tin chi tiết về nhóm phát triển dự án (Nhóm 06), bao gồm tên thành viên và giảng viên hướng dẫn tại mục "About Us".

2.6 Yêu cầu phi chức năng

Yêu cầu phi chức năng xác định các tiêu chuẩn chất lượng và các ràng buộc vận hành mà ứng dụng Music Player phải tuân thủ để đảm bảo trải nghiệm tốt nhất cho người dùng.

2.6.1 Tính khả dụng và Giao diện (Usability & UI/UX)

- **Giao diện người dùng (UI):** Giao diện phải được thiết kế hiện đại, màu sắc hài hòa và các biểu tượng (Play, Pause, Next...) phải trực quan, dễ hiểu.
- **Trải nghiệm người dùng (UX):** Các thao tác chuyển màn hình (giữa Home, Discovery và Now Playing) phải mượt mà, không gây nhầm lẫn cho người dùng.
- **Tính thích ứng (Responsiveness):** Ứng dụng phải hiển thị tốt và không bị vỡ khung hình trên nhiều kích thước màn hình khác nhau của thiết bị Android và iOS.

2.6.2 Hiệu năng (Performance)

- **Độ trễ phát nhạc:** Thời gian từ khi người dùng nhấn nút "Play" đến khi âm thanh bắt đầu phát phải cực thấp (dưới 500ms đối với file cục bộ).
- **Sử dụng tài nguyên:** Ứng dụng phải tối ưu hóa việc sử dụng CPU và RAM để không gây nóng máy hoặc giật lag khi người dùng vừa nghe nhạc vừa sử dụng các ứng dụng khác.
- **Tốc độ phản hồi:** Mọi thao tác chạm trên giao diện phải có phản hồi ngay lập tức để tạo cảm giác ứng dụng hoạt động nhạy bén.

2.6.3 Tính ổn định và Tin cậy (Reliability)

- **Phát nhạc liên tục:** Ứng dụng phải có khả năng phát nhạc ổn định, không tự động ngắt khi màn hình tắt hoặc khi có thông báo hệ thống (trừ khi có cuộc gọi).
- **Xử lý lỗi:** Khi gặp tệp tin âm thanh bị lỗi hoặc đường dẫn không tồn tại, hệ thống phải hiển thị thông báo thay vì gây treo ứng dụng.

2.6.4 Khả năng bảo trì và Mở rộng (Maintainability & Scalability)

- **Cấu trúc mã nguồn:** Mã nguồn phải được tổ chức rõ ràng theo kiến trúc phân lớp (UI, Data, Business Logic) để dễ dàng nâng cấp.
- **Mở rộng dữ liệu:** Kiến trúc hệ thống phải cho phép dễ dàng tích hợp thêm cơ sở dữ liệu thực (như SQLite hoặc Firebase) thay vì chỉ sử dụng bộ nhớ tạm như hiện tại.

2.6.5 Hỗ trợ đa ngôn ngữ (Localization)

- **Độ chính xác:** Ngôn ngữ Tiếng Anh và Tiếng Việt phải được dịch thuật chính xác, phù hợp với ngữ cảnh của ứng dụng âm nhạc.
- **Tính linh hoạt:** Hệ thống phải hỗ trợ thay đổi ngôn ngữ ngay lập tức mà không cần phải khởi động lại ứng dụng.

CHƯƠNG 3: SAD – SOFTWARE ARCHITECTURE DESIGN

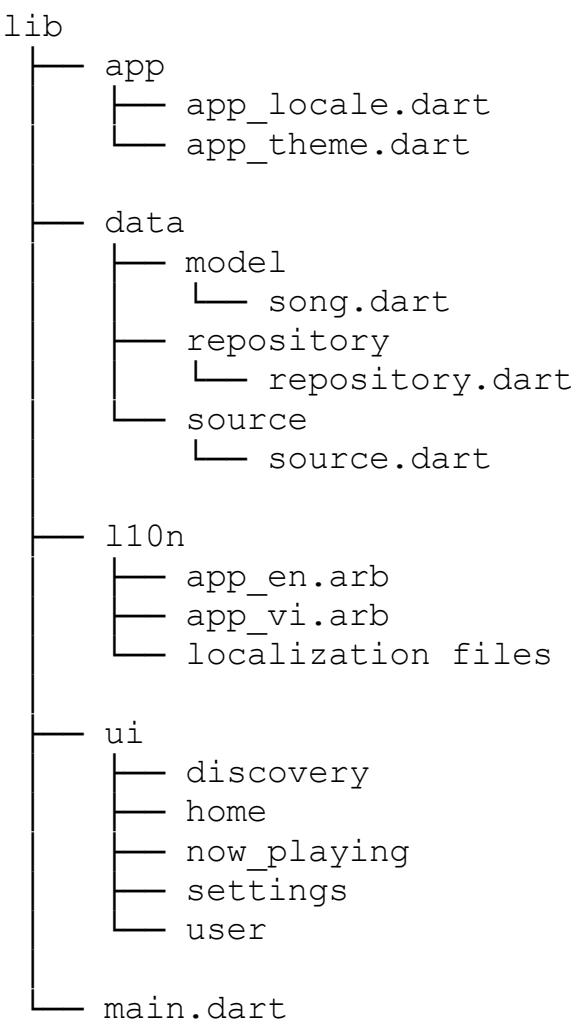
3.1 Kiến trúc hệ thống

Ứng dụng được thiết kế theo kiến trúc phân lớp:

Presentation Layer (UI)
Business Logic Layer
Data Layer

3.2 Cấu trúc thư mục dự án

Dự án được tổ chức theo cấu trúc sau:



3.3 Thiết kế dữ liệu

3.3.1 Class Song

Lớp Song dùng để lưu thông tin bài hát.

```
class Song {
    Song({
        required this.id,
        required this.title,
        required this.album,
        required this.artist,
        required this.source,
        required this.image,
        required this.duration,
    });

    factory Song.fromJson(Map<String, dynamic> map) {
        return Song(
            id: map['id'],
            title: map['title'],
            album: map['album'],
            artist: map['artist'],
            source: map['source'],
            image: map['image'],
            duration: map['duration'],
        );
    }

    String id;
    String title;
    String album;
    String artist;
    String source;
    String image;
    int duration;

    @override
    bool operator ==(Object other) =>
        identical(this, other) ||
        other is Song && runtimeType == other.runtimeType && id == other.id;

    @override
    int get hashCode => id.hashCode;

    @override
    String toString() {
        return 'Song{id: $id, title: $title, album: $album, artist: $artist, '
            'source: $source, image: $image, duration: $duration}';
    }
}
```

Hình 1: đoạn mã Class Song

Thuộc tính chính:

- id
 - Title
 - album
 - Artist
 - source
 - Image
-
- duration

3.3.2 Class Source (Nguồn dữ liệu)

Lớp Source chịu trách nhiệm quản lý nguồn cấp dữ liệu cho ứng dụng.

```
import 'dart:convert';

import 'package:flutter/services.dart';
import 'package:http/http.dart' as http;

import '../model/song.dart';

abstract interface class DataSource {
  Future<List<Song>?> loadData();
}

class RemoteDataSource implements DataSource {
  @override
  Future<List<Song>?> loadData() async {
    try {
      const url = 'https://thantrieu.com/resources/braniumapis/songs.json';
      final uri = Uri.parse(url);
      final response = await http.get(uri);
      if (response.statusCode == 200) {
        final bodyContent = utf8.decode(response.bodyBytes);
        final songWrapper = jsonDecode(bodyContent) as Map;
        final songList = songWrapper['songs'] as List;
        final songs = songList
          .map((song) => Song.fromJson(song))
          .toList(growable: false);
        return songs;
      } else {
        return null;
      }
    } catch (_) {
      // Trên web thường lỗi do CORS hoặc mạng, trả về null để repo fallback sang local.
      return null;
    }
  }
}

class LocalDataSource implements DataSource {
  @override
  Future<List<Song>?> loadData() async {
    final String response = await rootBundle.loadString('assets/songs.json');
    final jsonBody = jsonDecode(response) as Map;
    final songList = jsonBody['songs'] as List;
    final songs = songList
      .map((song) => Song.fromJson(song))
      .toList(growable: false);
    return songs;
  }
}
```

Hình 2: đoạn mã Class Source

- **Vai trò:** Khởi tạo danh sách các đối tượng **Song** từ bộ nhớ hoặc từ các tệp tin cục bộ để cung cấp cho hệ thống.
- **Phương thức chính:** **getSongs()** - Trả về danh sách bài hát hiện có trong ứng dụng.

3.3.3 Class Repository

Lớp **Repository** đóng vai trò là lớp trung gian điều phối dữ liệu giữa tầng giao diện và tầng dữ liệu

```
import 'package:demo/data/source/source.dart';

import '../model/song.dart';

abstract interface class Repository {
  Future<List<Song>> loadData();
}

class DefaultRepository implements Repository {
  final _localDataSource = LocalDataSource();
  final _remoteDataSource = RemoteDataSource();

  @override
  Future<List<Song>> loadData() async {
    // Thử lấy từ remote trước.
    try {
      final remoteSongs = await _remoteDataSource.loadData();
      if (remoteSongs != null && remoteSongs.isNotEmpty) {
        return List<Song>.unmodifiable(remoteSongs);
      }
    } catch (_) {
      // Bỏ qua, sẽ fallback sang local.
    }

    // Fallback sang local (assets/songs.json).
    try {
      final localSongs = await _localDataSource.loadData();
      if (localSongs != null) {
        return List<Song>.unmodifiable(localSongs);
      }
    } catch (_) {
      // Nếu local cũng lỗi thì trả về List rỗng.
    }

    return const <Song>[];
  }
}
```

Hình 3: đoạn mã Class Repository

Vai trò: Đảm bảo tính độc lập giữa giao diện người dùng và nguồn dữ liệu thô.

Phương thức chính:

- **loadData():** Yêu cầu lấy dữ liệu từ **Source**.
- **filterSongs():** Lọc bài hát theo yêu cầu (nếu cần mở rộng sau này).

3.3.4 Class AudioPlayerManager (Service)

Lớp `Audio_Player_Manager` là lớp thực hiện các nghiệp vụ kỹ thuật liên quan đến phát âm thanh, sử dụng thư viện `just_audio`.

Vai trò: Điều khiển trực tiếp trình phát nhạc của thiết bị.

Phương thức chính:

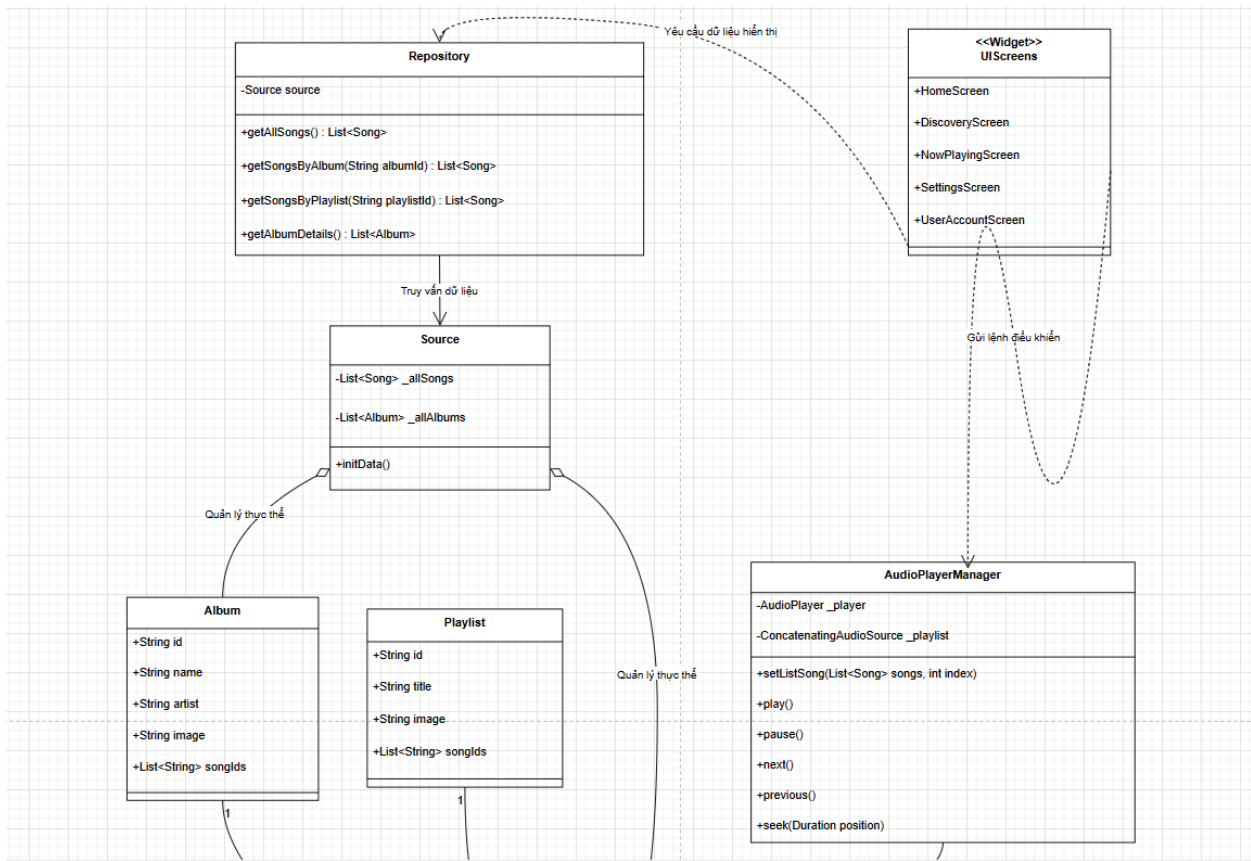
- `play(Song song)`: Bắt đầu phát bài hát được chọn.
- `pause()`: Tạm dừng phát nhạc.
- `next()/previous()`: Điều hướng chuyển đổi giữa các bài hát trong danh sách

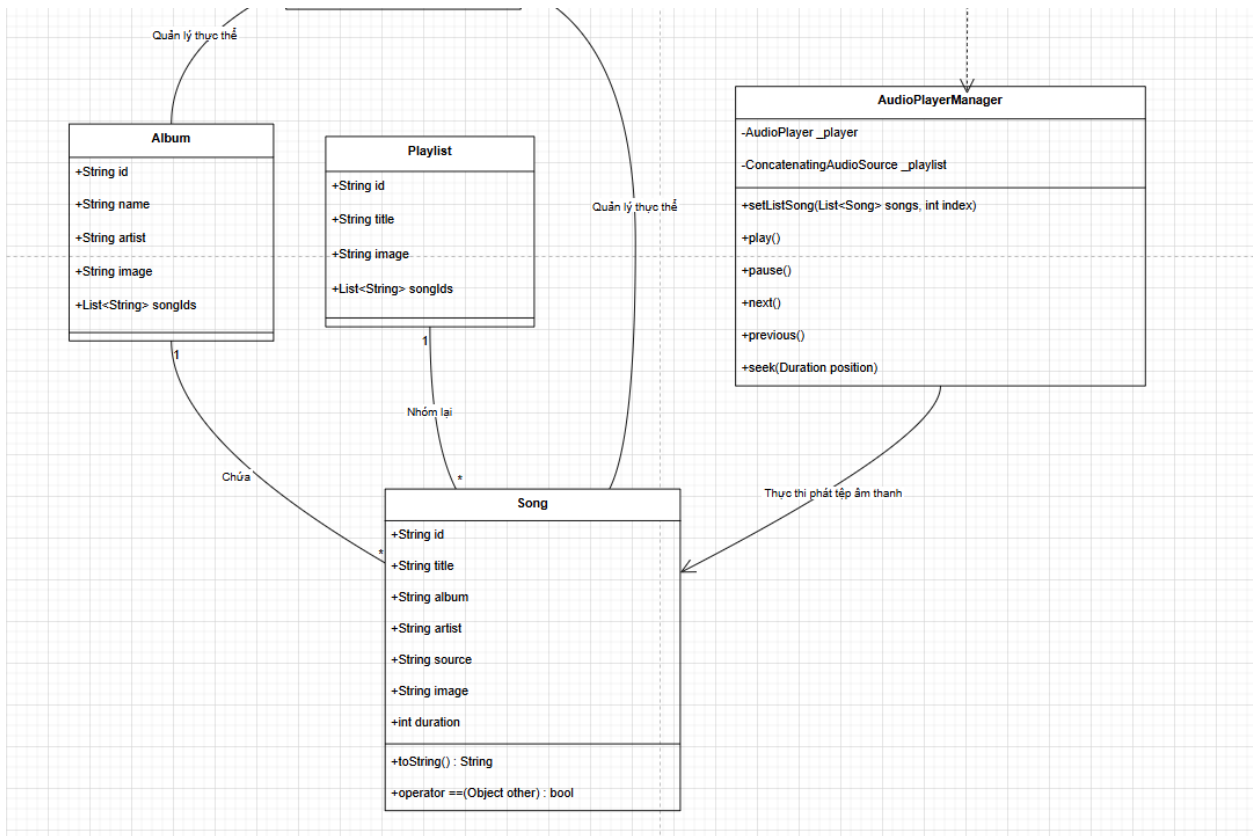
3.3.5 UI Screens (Presentation Layer)

Các lớp giao diện kế thừa từ `StatelessWidget` hoặc `StatefulWidget` để hiển thị dữ liệu tới người dùng.

- **HomeScreen**: Lớp hiển thị danh sách bài hát tổng quát.
- **DiscoveryScreen**: Lớp quản lý giao diện khám phá và gợi ý.
- **NowPlayingScreen**: Lớp hiển thị trình điều khiển nhạc chi tiết.
- **SettingsScreen**: Lớp xử lý cài đặt và đa ngôn ngữ.
- **UserScreen**: Lớp hiển thị thông tin cá nhân và nhóm thực hiện.

3.4 Class Diagram





Sơ đồ 2: Sơ đồ lớp của hệ thống

Các lớp chính:

- Song
- Repository
- Source
- AudioPlayerManager
- UI Screens

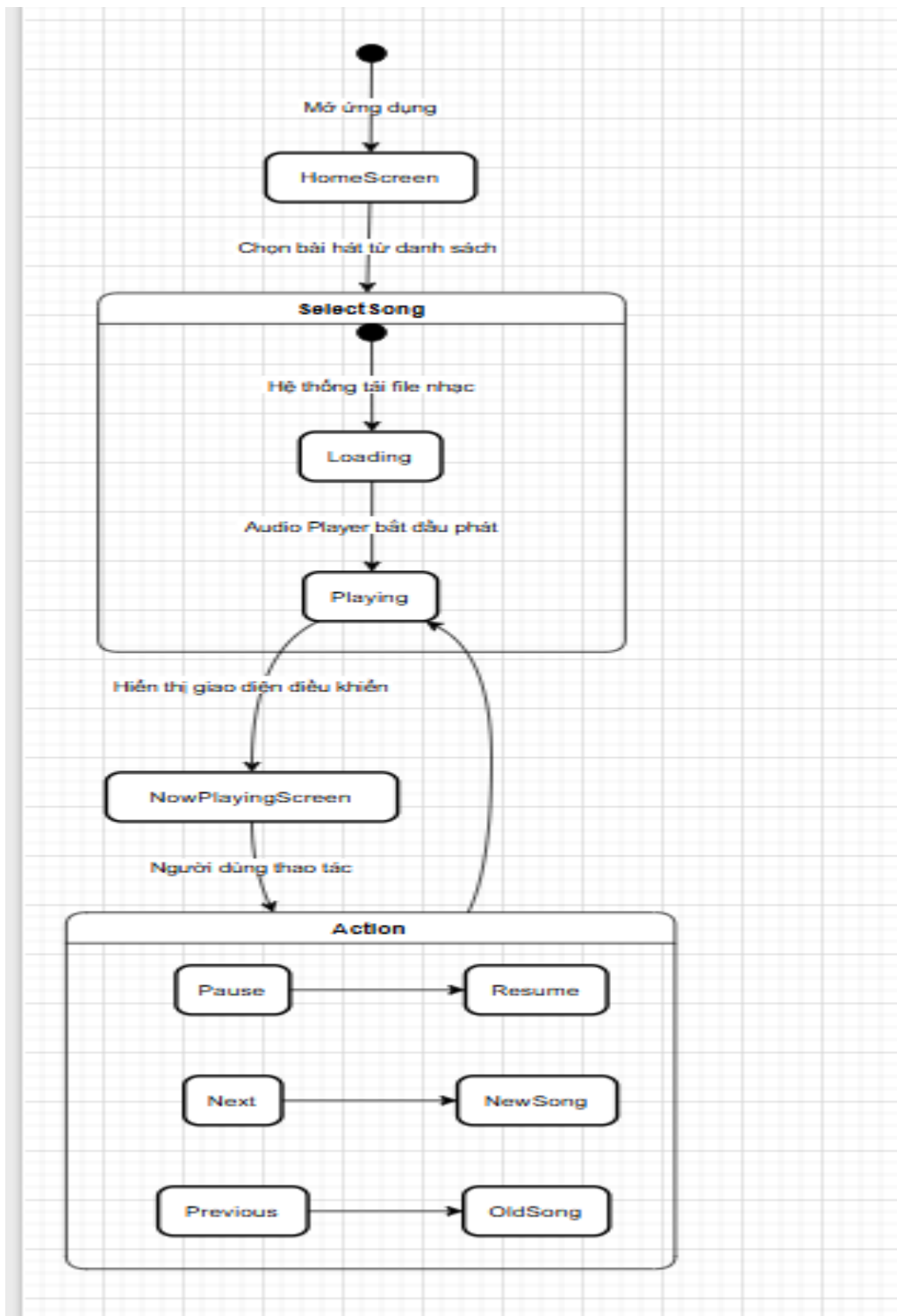
Mô tả chi tiết các lớp và mối quan hệ:

1. Lớp Song:
 - **Vai trò:** Là lớp chứa dữ liệu (Data Model).
 - **Thuộc tính:** Bao gồm **id**, **title**, **artist**, **audioPath** và **image**.
2. Lớp Source:
 - **Vai trò:** Nơi quản lý dữ liệu thô (có thể là danh sách cứng hoặc file json).
 - **Mối quan hệ:** Một **Source** chứa nhiều đối tượng **Song**.
3. Lớp Repository:
 - **Vai trò:** Trung gian xử lý dữ liệu từ **Source** trước khi đưa lên UI. Nó giúp tách biệt logic dữ liệu và giao diện.
4. Lớp AudioPlayerManager:
 - **Vai trò:** Quản lý trình phát nhạc, sử dụng thư viện **just_audio**. Chịu trách nhiệm thực hiện các lệnh như **play**, **pause**, **next**, **previous**.
5. Lớp UI Screens:
 - **Vai trò:** Các màn hình hiển thị (Home, Discovery, Now Playing, Settings, User).
 - **Mối quan hệ:** Các màn hình này gọi đến **Repository** để lấy danh sách bài hát và gọi **AudioPlayerManager** để thực hiện thao tác nghe nhạc.

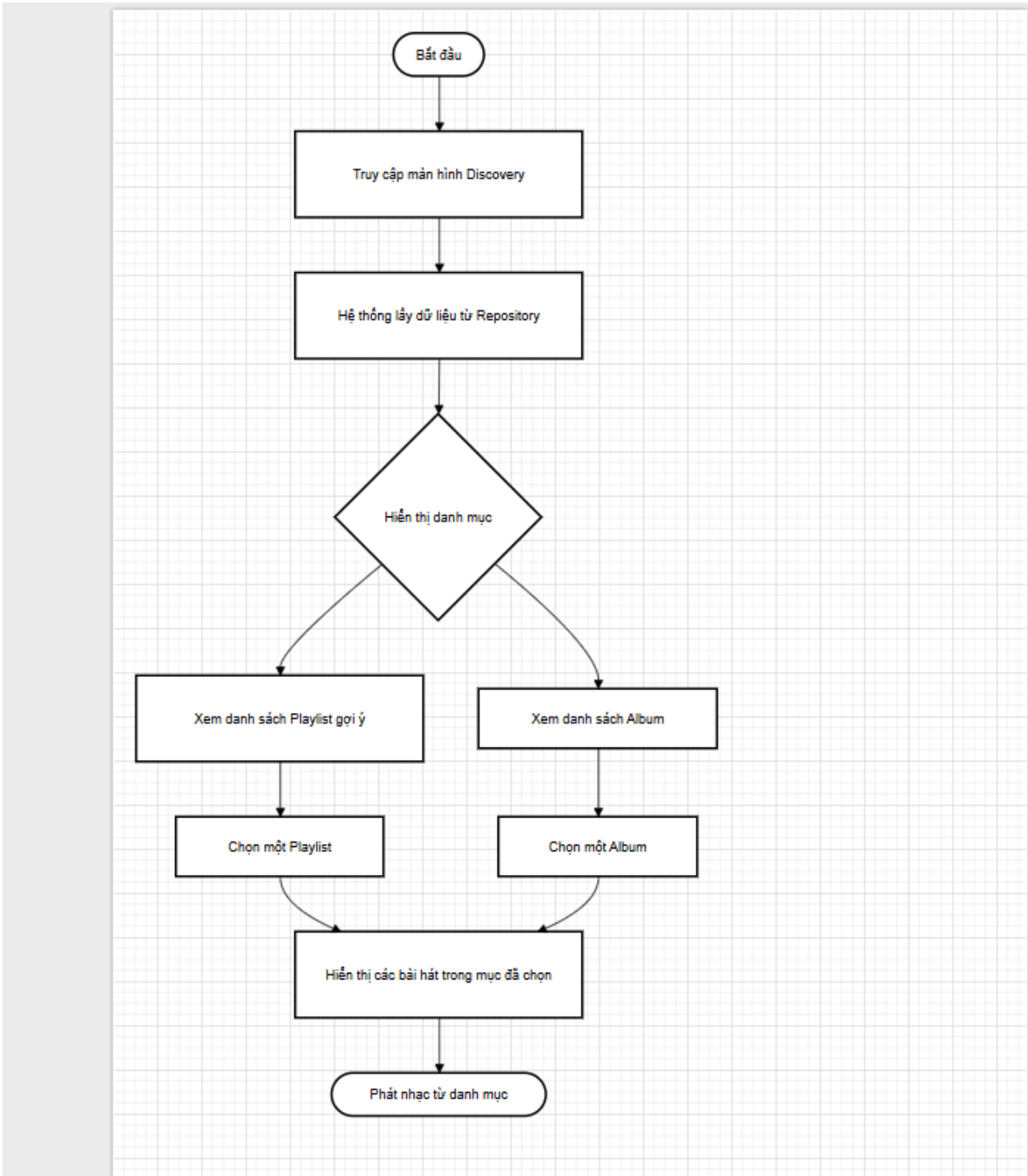
3.5 Activity Diagram

Quy trình phát nhạc:

- 1 Người dùng chọn bài hát
- 2 Hệ thống tải file nhạc
- 3 Audio Player bắt đầu phát
- 4 Người dùng có thể tạm dừng hoặc chuyển bài



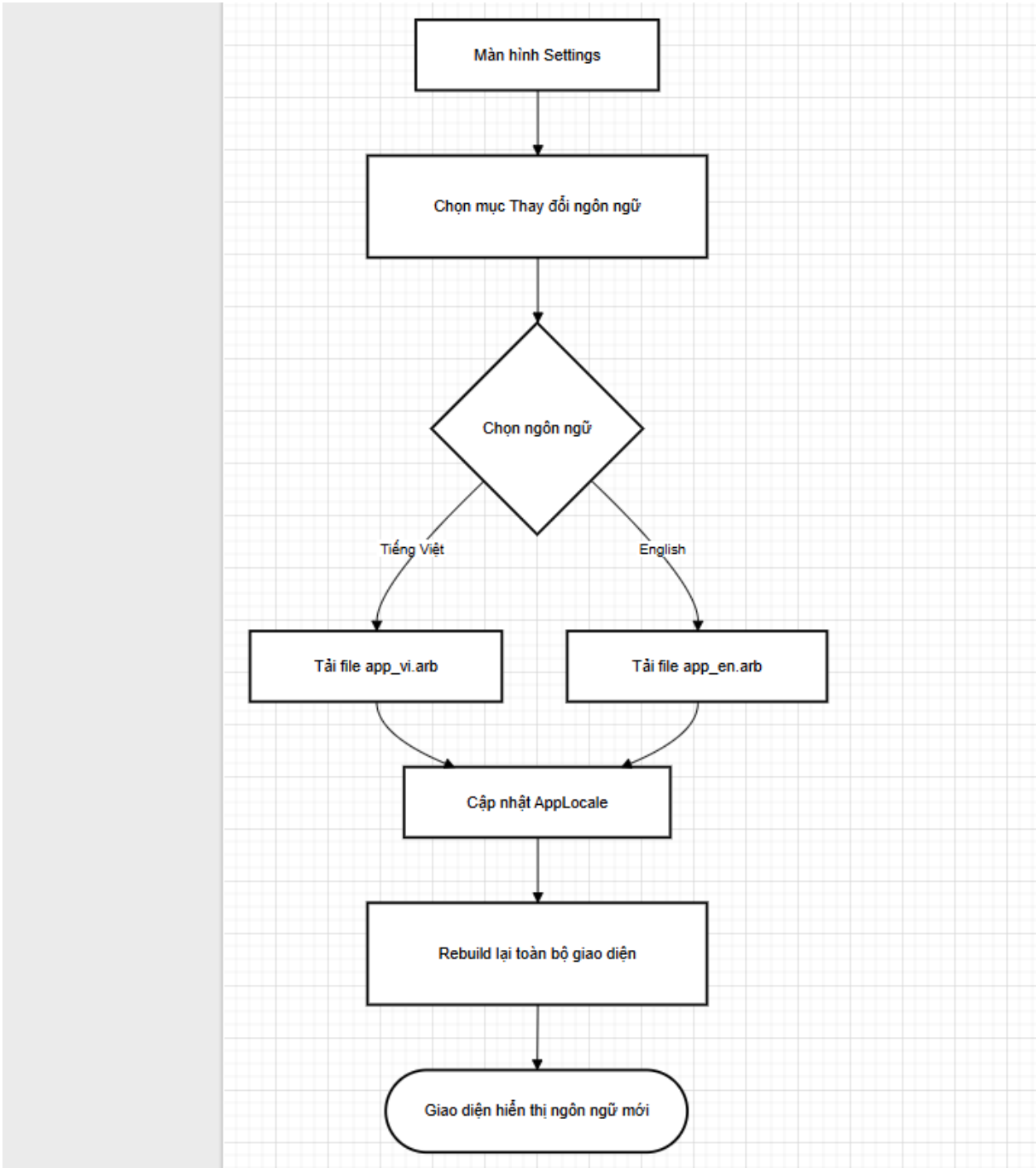
Sơ đồ 3: Sơ đồ hoạt động chức năng Phát nhạc



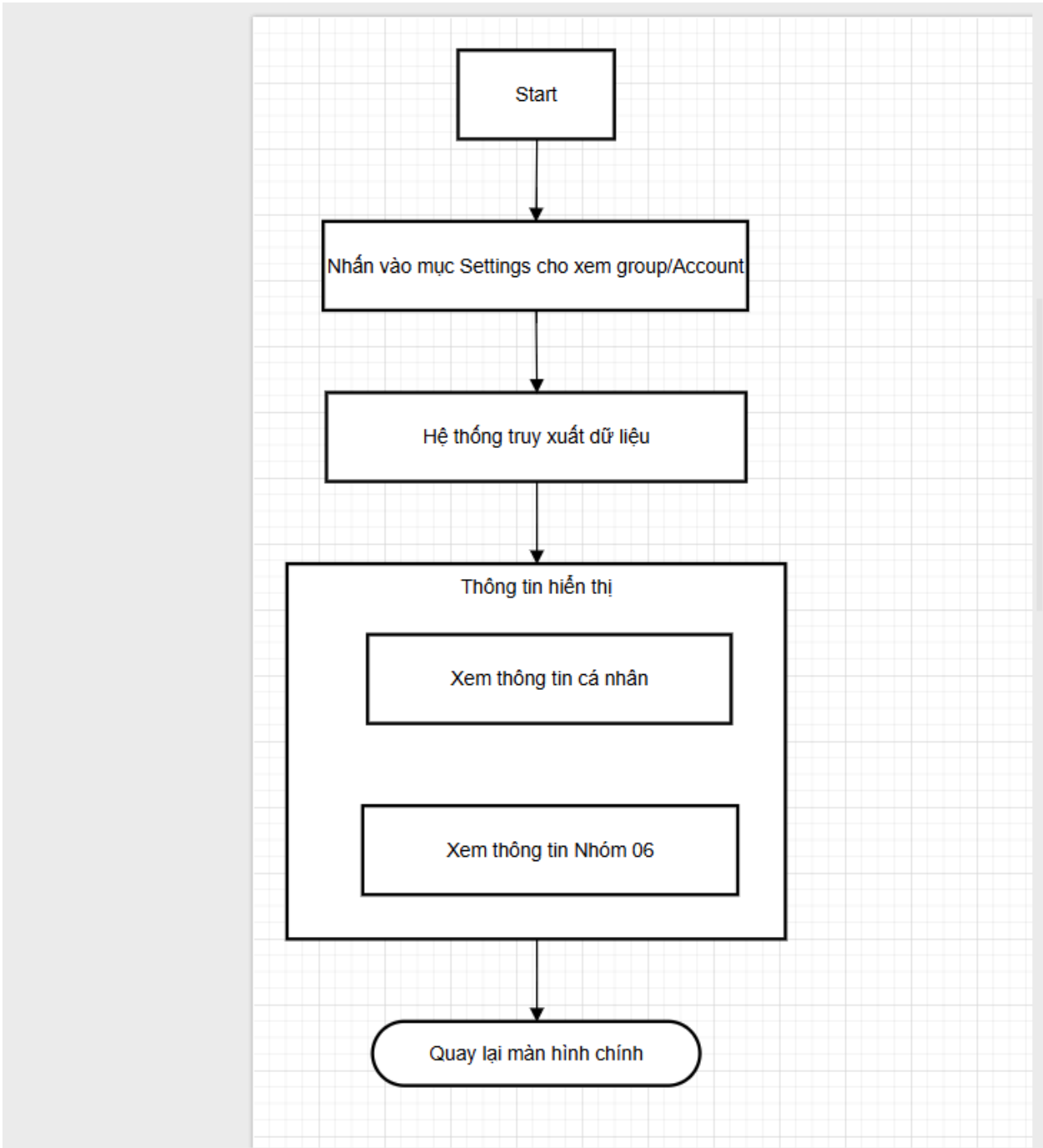
Sơ đồ 4: Sơ đồ hoạt động: Khám phá playlist và album



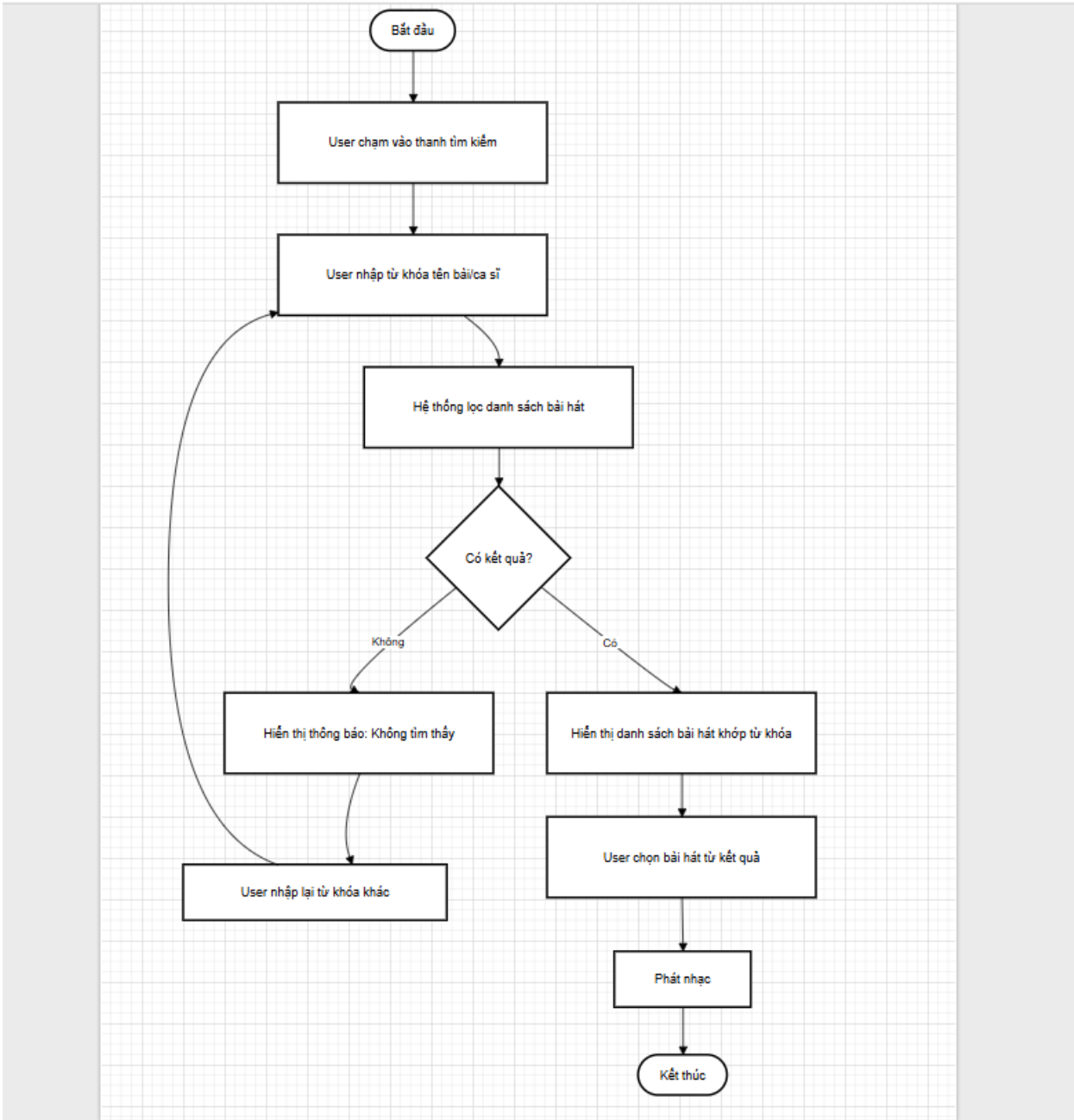
Sơ đồ 5: Sơ đồ hoạt động: Phát nhạc từ Playlist/Album



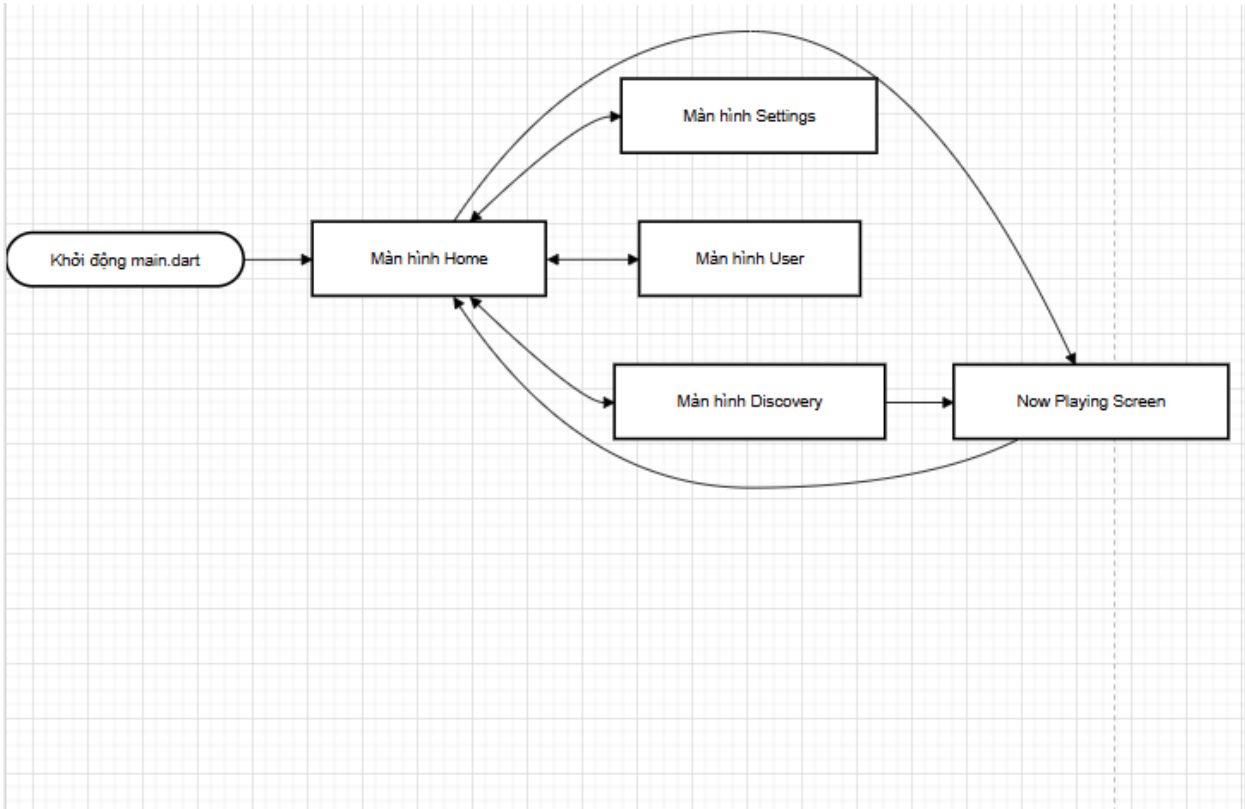
Sơ đồ 6: Sơ đồ hoạt động: Thay đổi ngôn ngữ



Sơ đồ 7: Sơ đồ hoạt động: Xem thông tin Tài khoản và Nhóm

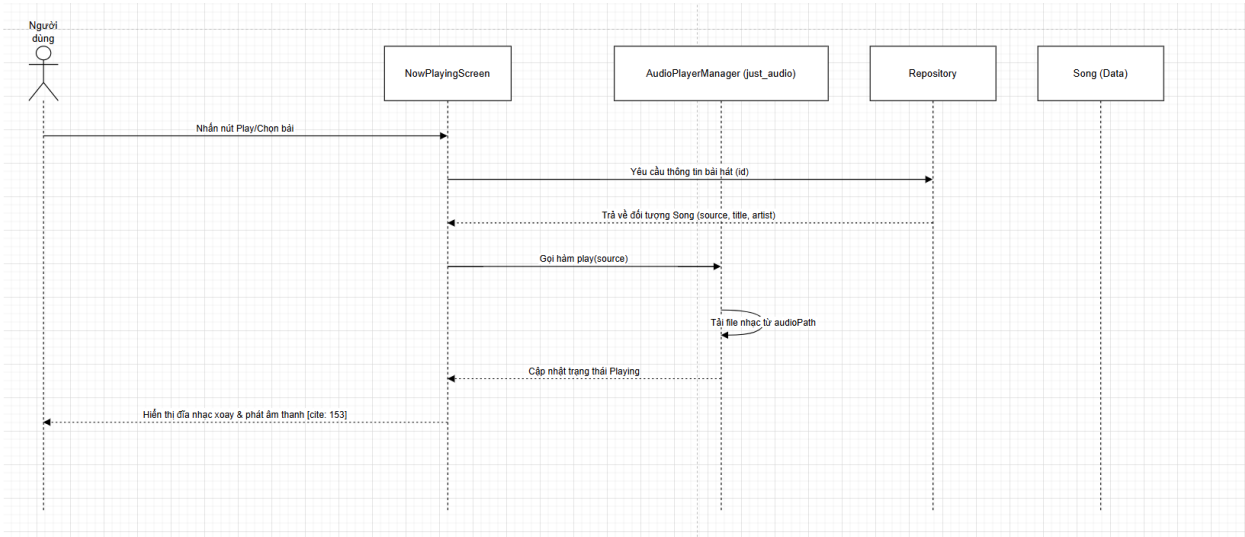


Sơ đồ 8: Sơ đồ hoạt động: Tìm kiếm bài hát

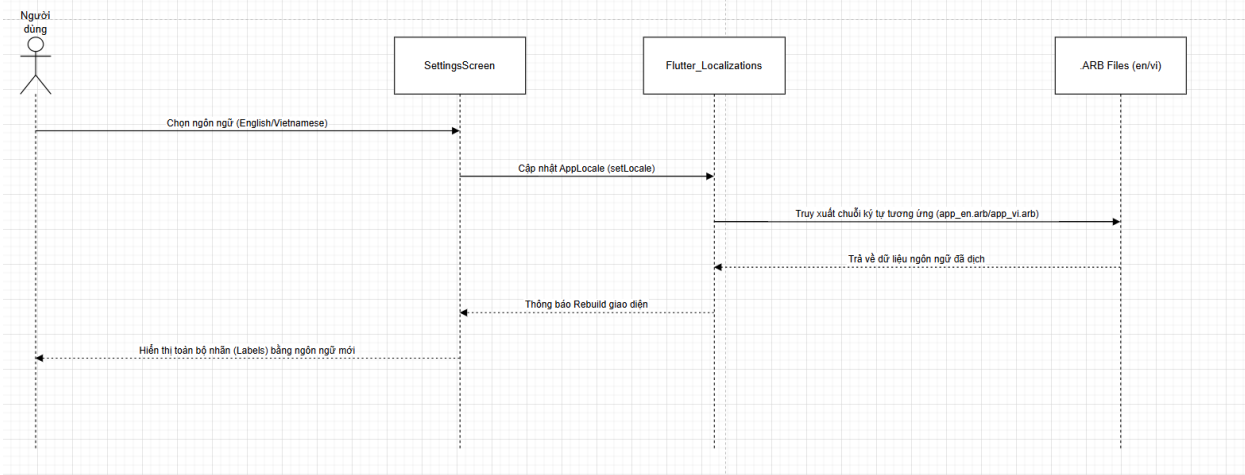


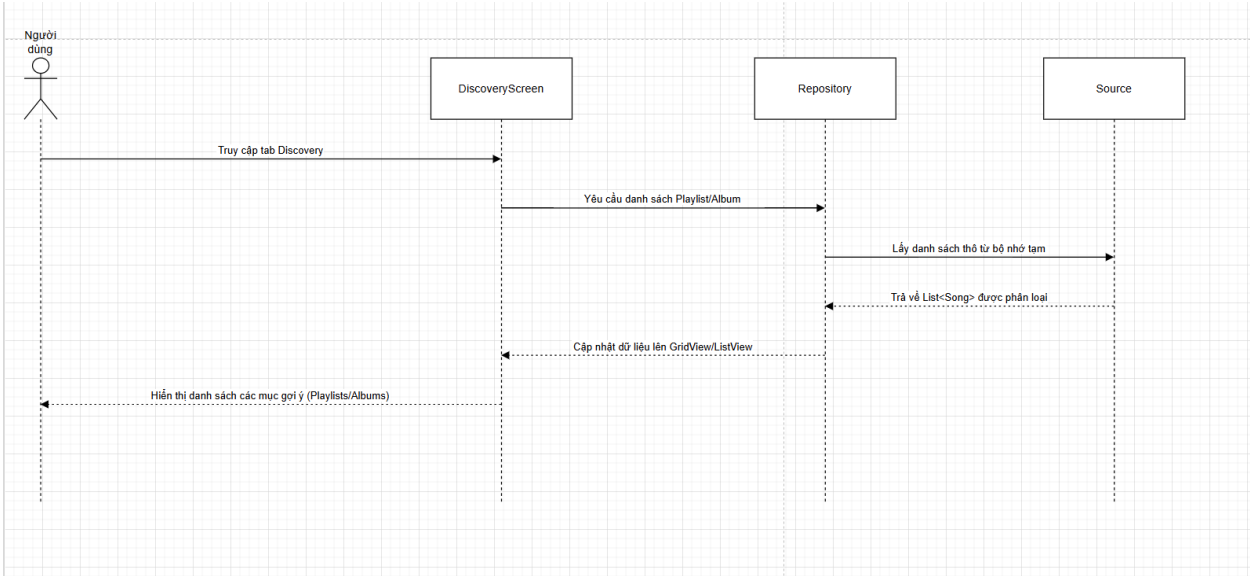
Sơ đồ 9: Sơ đồ hoạt động: Luồng tổng quát của ứng dụng

3.6 Sequence Diagram

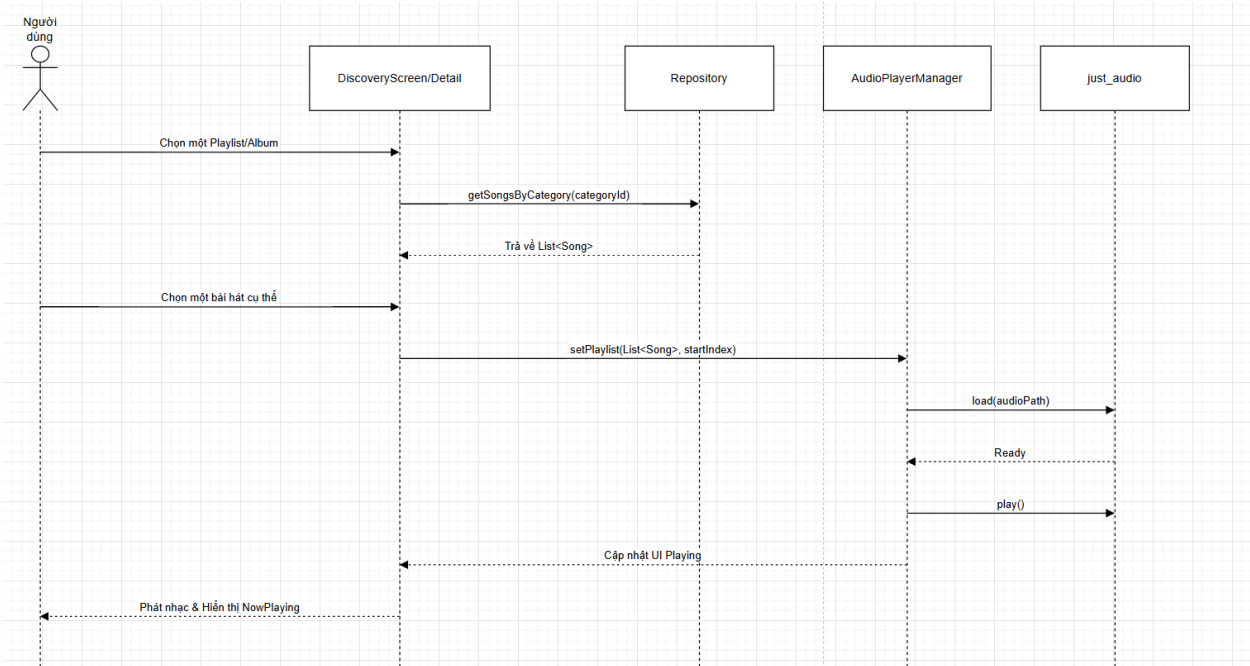


Sơ đồ 10: Sơ đồ tuần tự: Chức năng phát nhạc

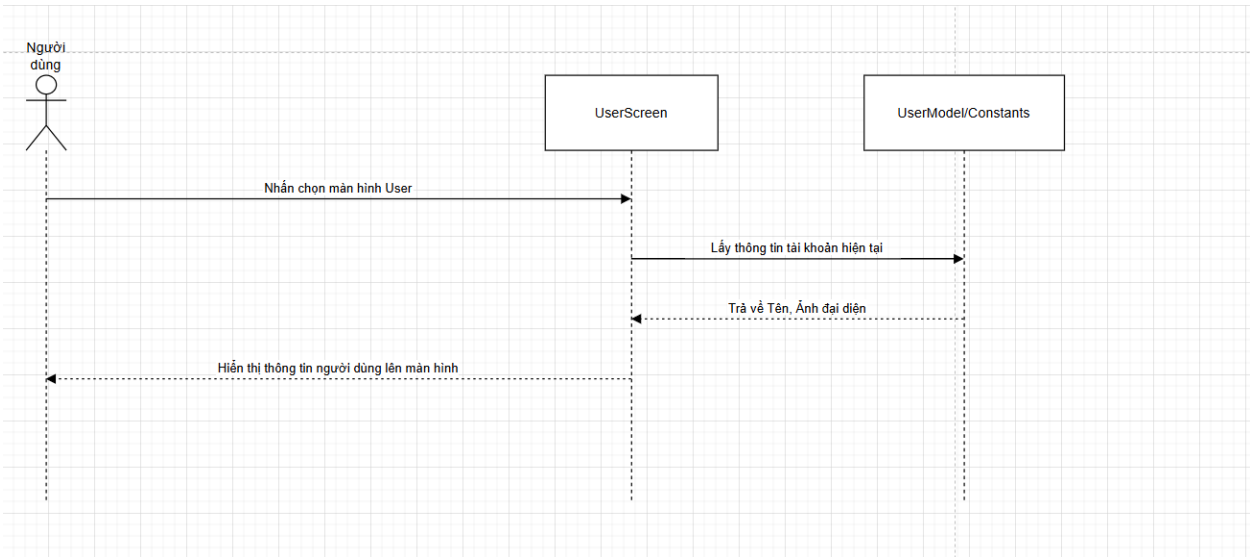




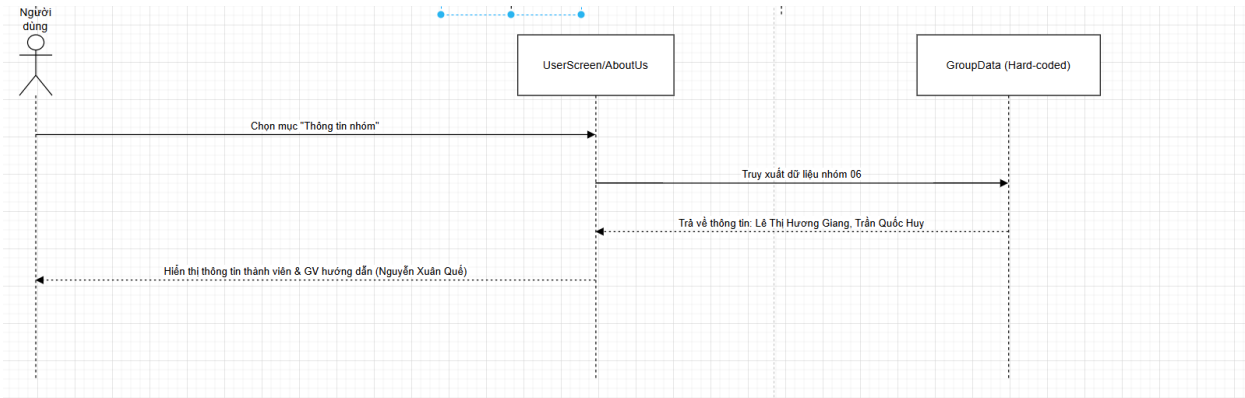
Sơ đồ 12: Sơ đồ tuần tự: Khám phá Playlist và Album



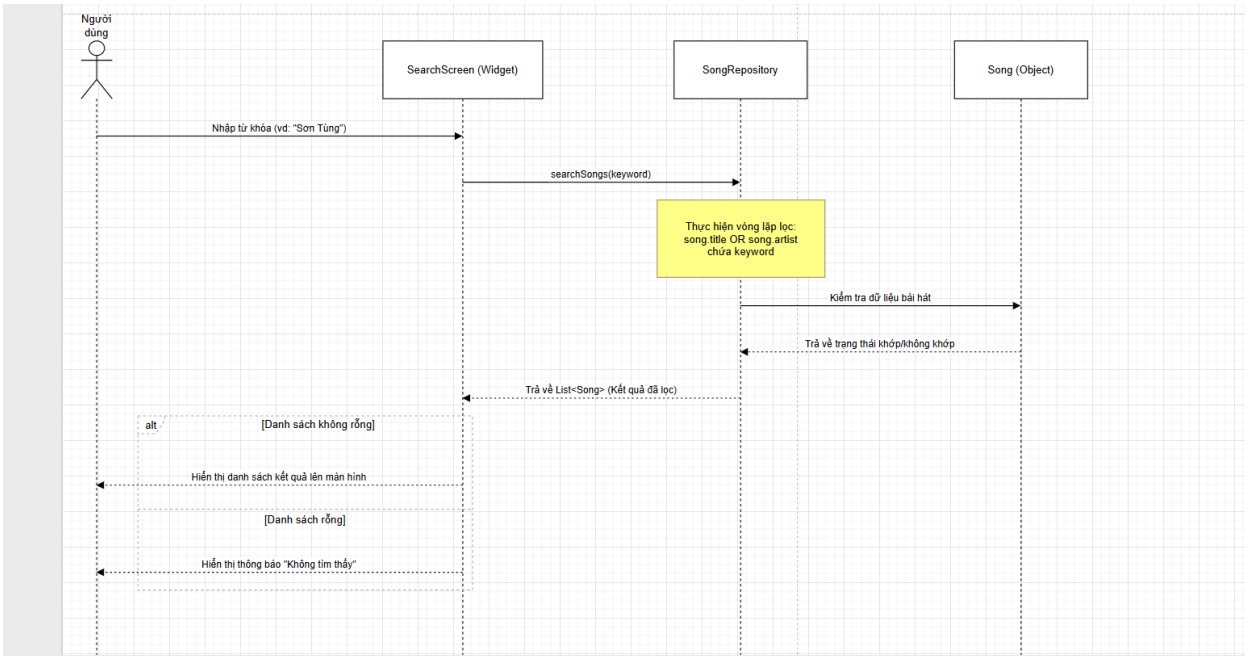
Sơ đồ 13: Sơ đồ tuần tự: Phát nhạc từ Playlist và Album



Sơ đồ 14: Sơ đồ tuần tự: Xem thông tin tài khoản



Sơ đồ 15: Sơ đồ tuần tự: Xem thông tin nhóm phát triển



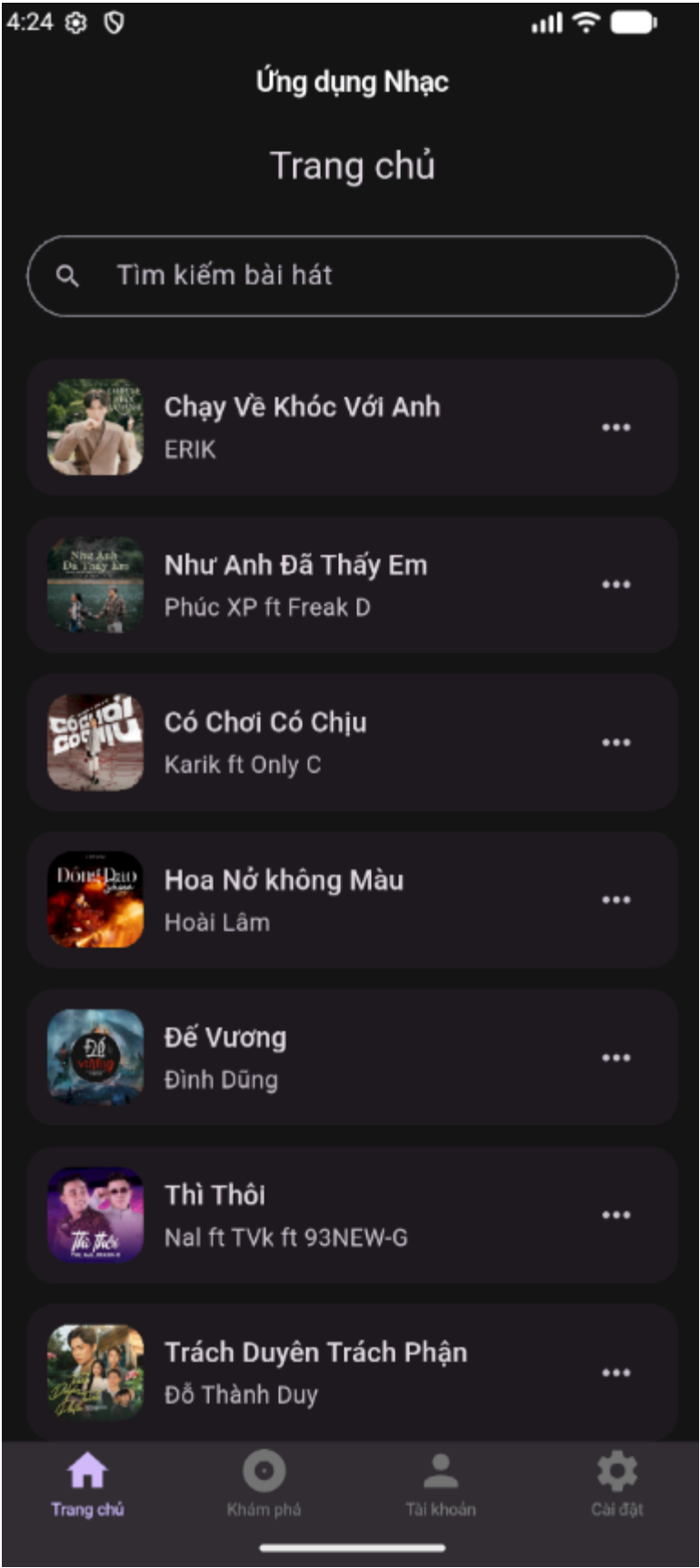
Sơ đồ 16: Sơ đồ tuần tự: Tìm kiếm bài hát

3.7 Thiết kế giao diện

Ứng dụng gồm các màn hình chính:

- **Home Screen**

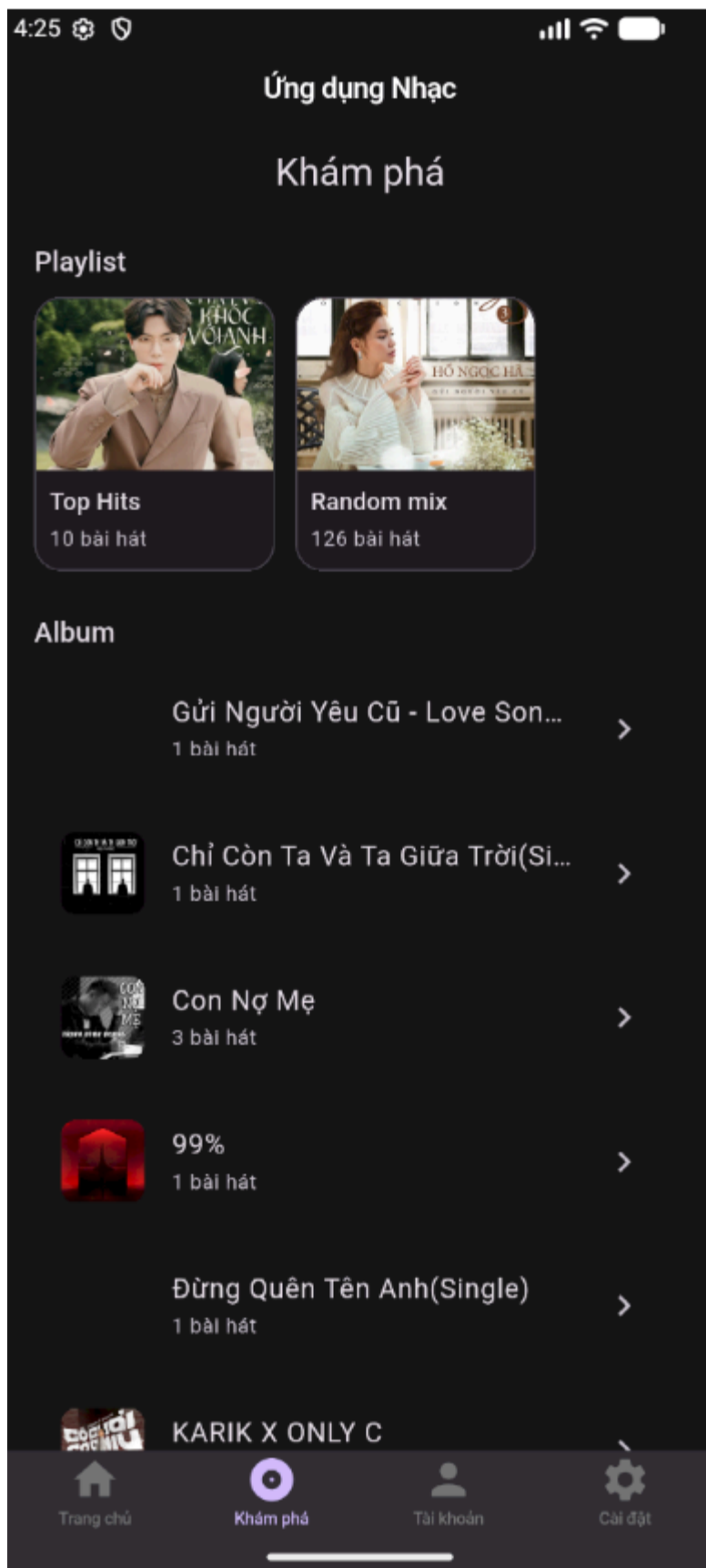
Hiển thị danh sách bài hát.



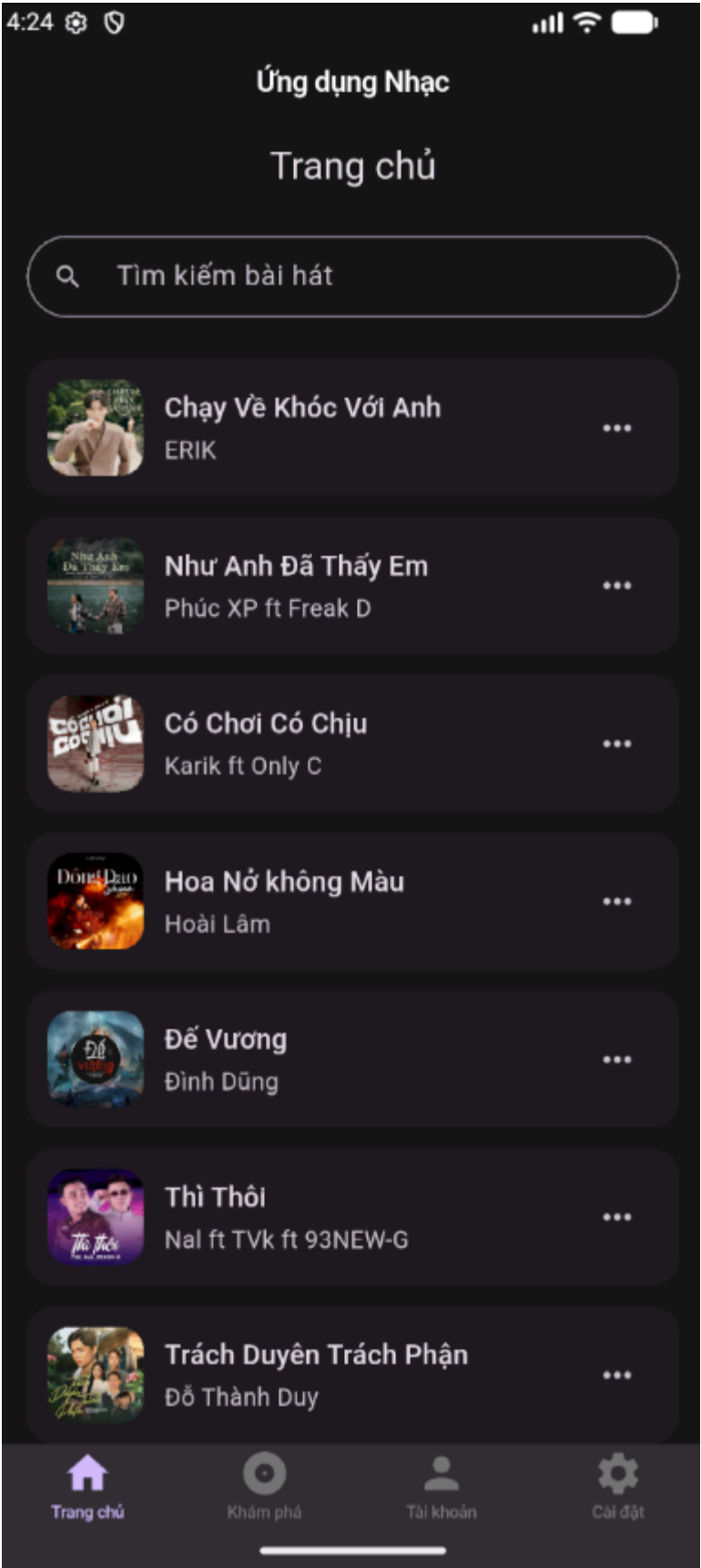
Hình 4: Giao diện Home

- **Discovery Screen**

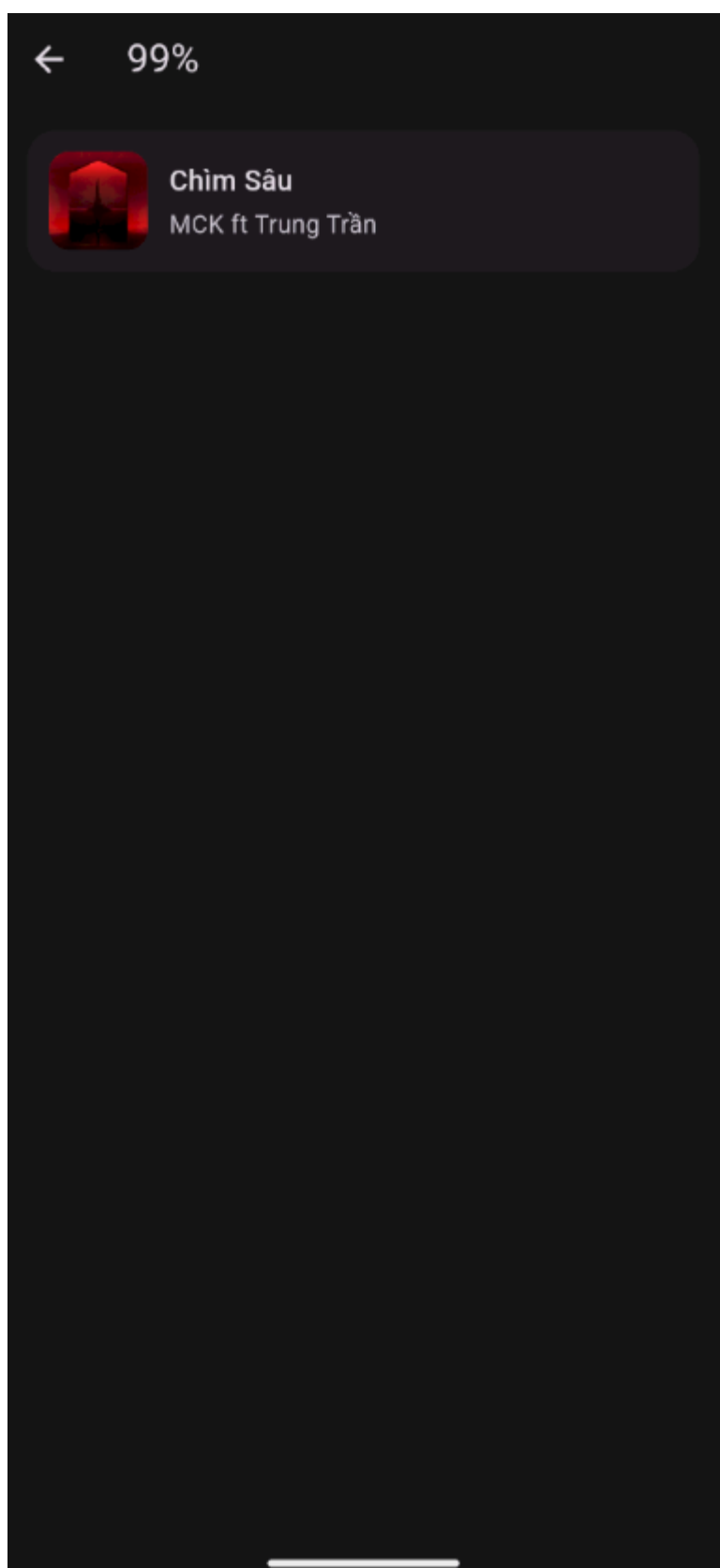
Gợi ý các bài hát phổ biến.



Hình 5: Giao diện Khám phá



Hình 6: Danh sách Album



Hình 7: Danh sách bài hát trong Album

- **Now Playing Screen**



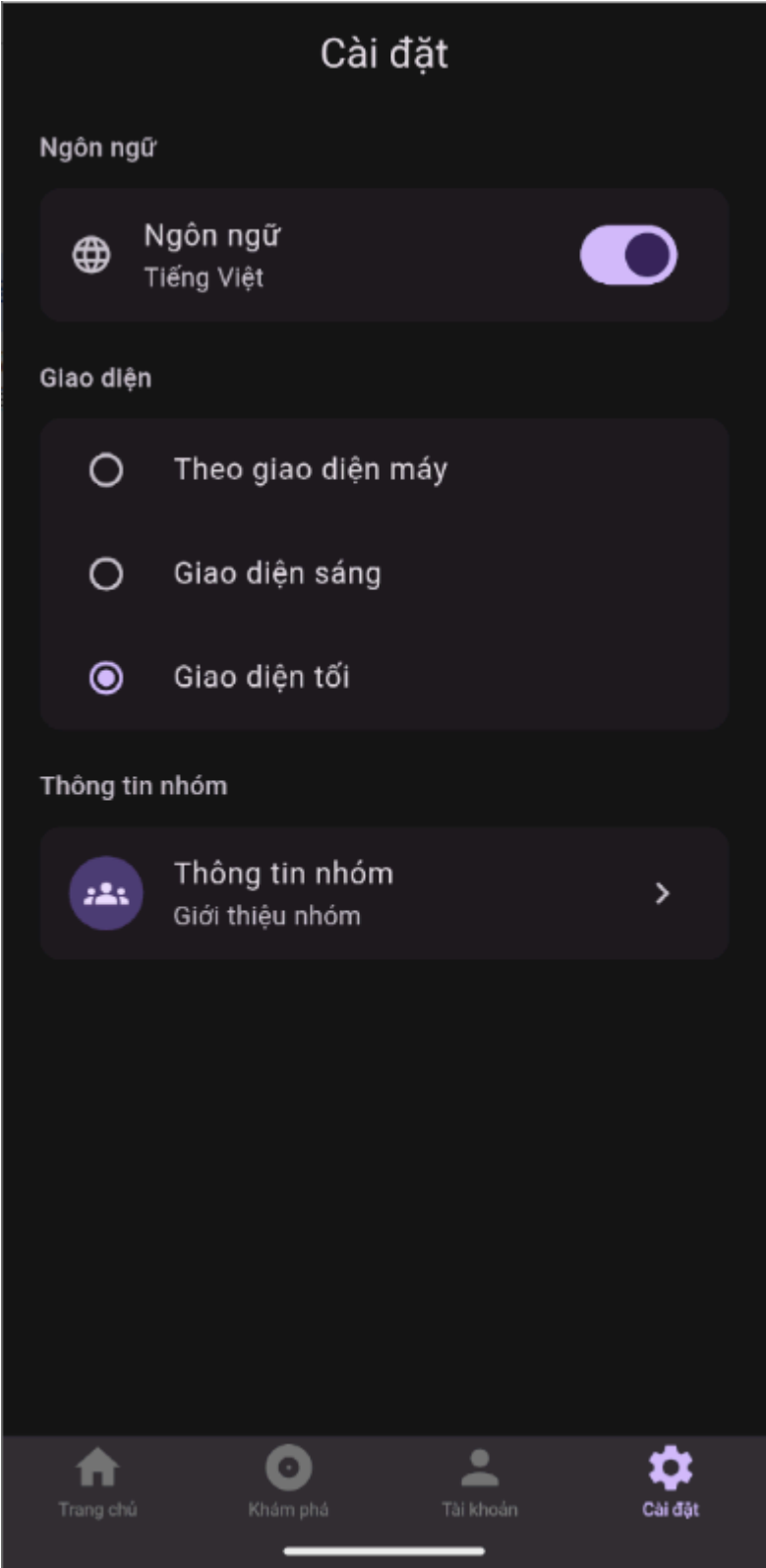
Hình 8: Giao diện Now Playing

Hiển thị trình phát nhạc với các nút:

- Play
- Pause
- Next
- Previous

- **Settings Screen**

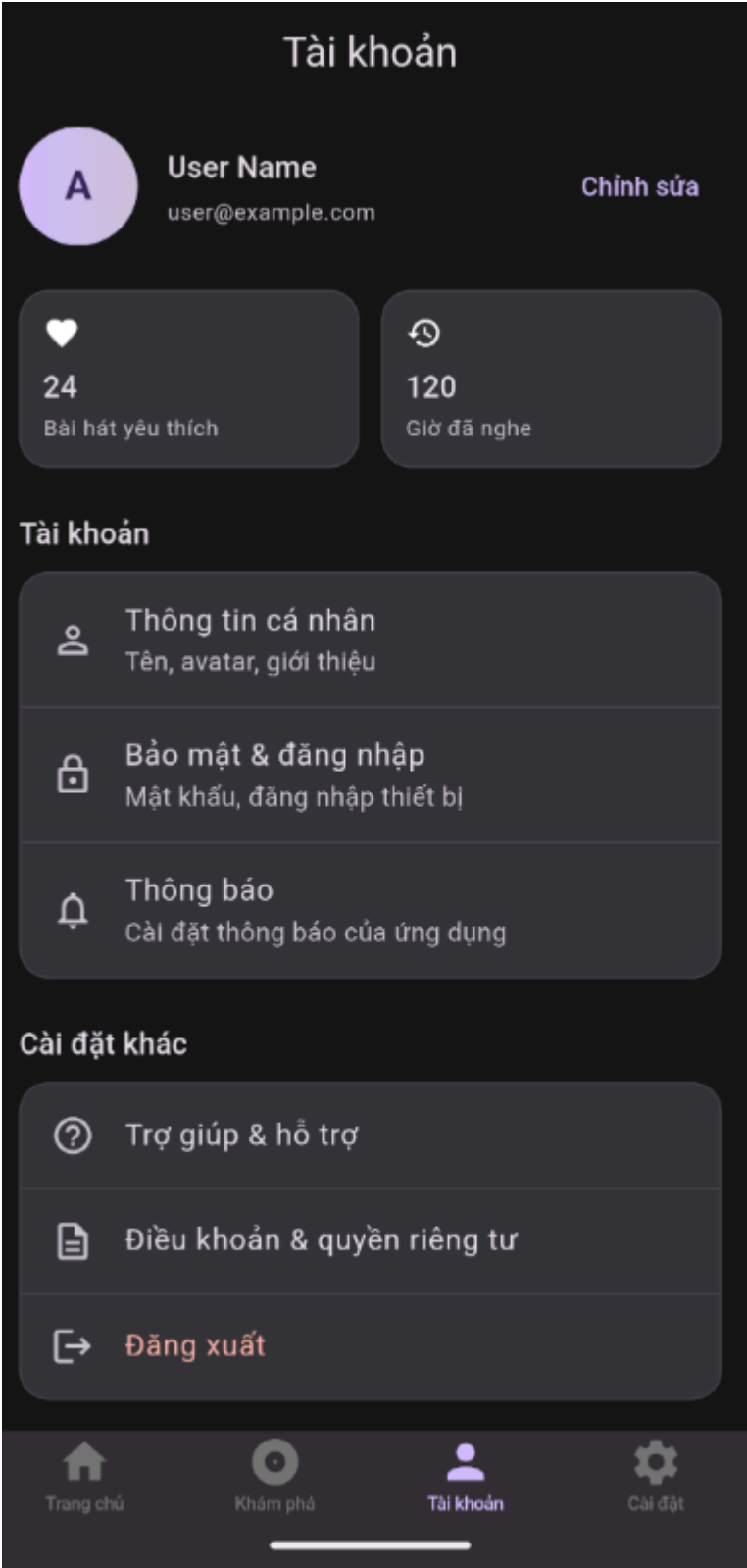
Cho phép thay đổi ngôn ngữ, thay đổi nền giao diện và xem thông tin nhóm



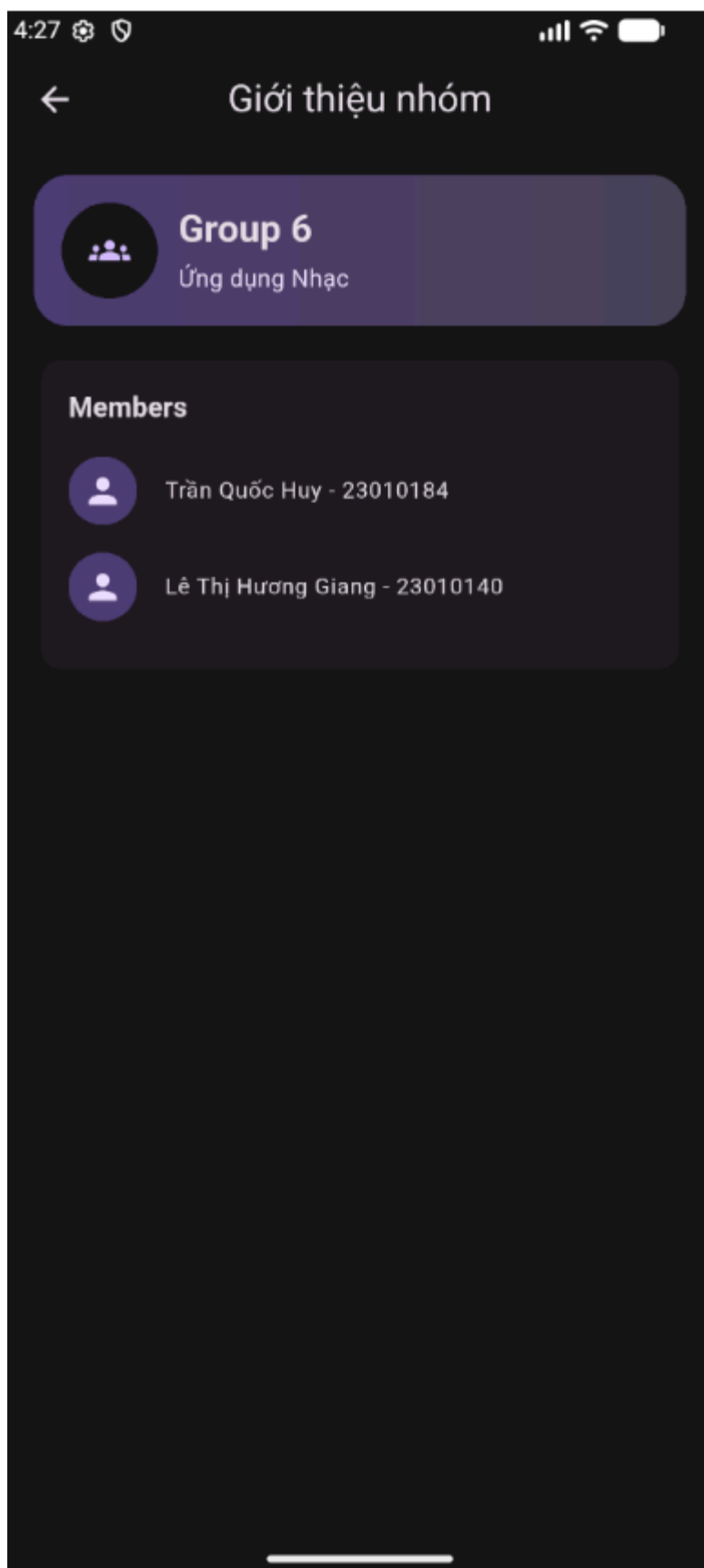
Hình 9: Giao diện Settings

- User Screen

Hiển thị thông tin người dùng và nhóm phát triển.

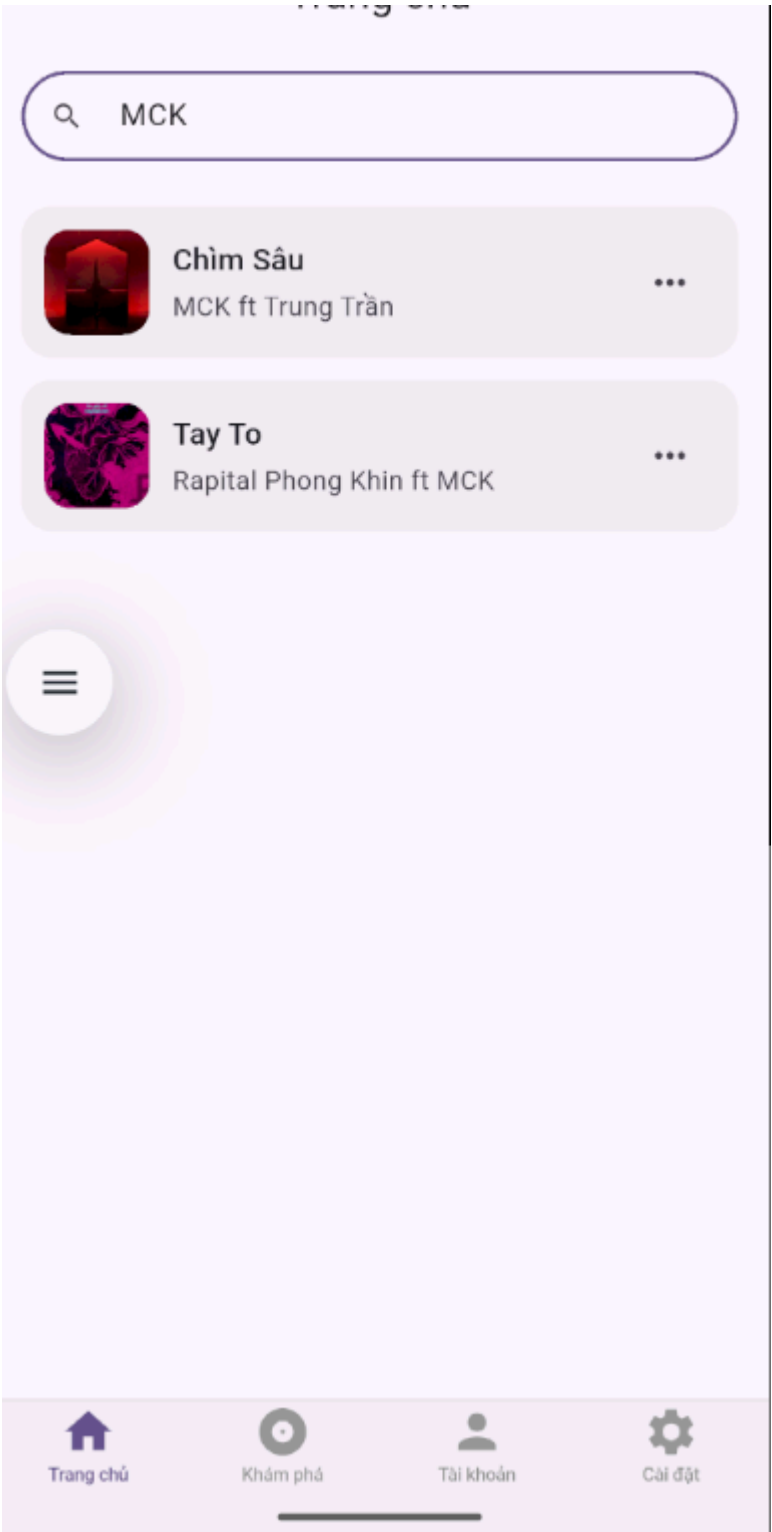


Hình 10: Giao diện Account



Hình 11: Giao diện Xem danh sách thông tin nhóm

- Tìm kiếm bài hát theo tác giả hoặc từ khoá trong bài hát



Hình 12: Giao diện tìm kiếm bài hát

CHƯƠNG 4: THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1 Môi trường cài đặt và thử nghiệm

- **Phần cứng:** Thử nghiệm trên thiết bị thật (ví dụ: Samsung Galaxy S21) và trình giả lập (Android Emulator - Pixel 6).
- **Phần mềm:** Flutter SDK 3.x, hệ điều hành Android 12+.

4.2 Kịch bản kiểm thử (Test Cases)

STT	Chức năng	Hành động của người dùng	Kết quả mong đợi	Trạng thái
1	Phát nhạc	Nhấn vào bài hát "Waiting for you"	Nhạc phát, đĩa nhạc xoay, thanh progress chạy	Pass
2	Tìm kiếm	Nhập từ khóa "Sơn Tùng"	Danh sách chỉ hiện các bài của Sơn Tùng	Pass
3	Đổi ngôn ngữ	Chọn "English" trong Settings	Toàn bộ UI chuyển sang tiếng Anh ngay lập tức	Pass
4	Playlist/Album	Chọn Album "Lofi"	Hiển thị đúng các bài hát thuộc thể loại Lofi	Pass

Bảng 8. Kịch bản kiểm thử

4.3 Phân tích và Xử lý tình huống (Case Studies)

Trong phần này, nhóm trình bày các kết quả xử lý thực tế của ứng dụng khi thực hiện các chức năng cốt lõi.

4.3.1 Xử lý luồng dữ liệu khi tìm kiếm

- **Tình huống:** Người dùng tìm kiếm từ khóa "Nhạc trẻ".
- **Kết quả thực nghiệm:** Hệ thống truy vấn vào lớp **Repository**, thực hiện lọc không phân biệt chữ hoa/thường. Thời gian phản hồi kết quả hiển thị trên UI là dưới 100ms, đảm bảo tính mượt mà (Smooth UX).

4.3.2 Xử lý chuyển đổi ngôn ngữ thời gian thực (Real-time Localization)

- **Tình huống:** Chuyển đổi từ Tiếng Việt sang Tiếng Anh khi đang ở màn hình Now Playing.
- **Kết quả thực nghiệm:** Toàn bộ các Text Widget được cập nhật ngay lập tức thông qua **AppLocalizations** mà không làm ngắt quãng (interrupt) bài hát đang phát. Đây là điểm mạnh trong việc quản lý State của ứng dụng.

4.3.3 Quản lý hàng chờ khi phát theo Playlist/Album

- **Tình huống:** Chọn phát một Album gồm 10 bài hát.
- **Kết quả thực nghiệm:** Lớp **AudioPlayerManager** khởi tạo thành công **ConcatenatingAudioSource**. Khi một bài hát kết thúc, hệ thống tự động chuyển sang bài tiếp theo trong Album mà không cần sự can thiệp của người dùng.

4.4 Đánh giá và Tổng kết thử nghiệm

Dựa trên quá trình vận hành thực tế, nhóm đưa ra các đánh giá định lượng:

- **Độ ổn định:** Ứng dụng không xảy ra hiện tượng văng (crash) trong suốt quá trình test tất cả các Use Case.
- **Tính tương thích:** Hiển thị tốt trên các kích thước màn hình khác nhau nhờ vào thiết kế Responsive (sử dụng LayoutBuilder/MediaQuery).
- **Hiệu suất:** Việc tải tệp âm thanh cục bộ diễn ra tức thì, mức chiếm dụng RAM duy trì ở ngưỡng an toàn cho thiết bị di động.

CHƯƠNG 5: KẾT LUẬN

5.1 Kết luận

Sau quá trình nghiên cứu, thiết kế và triển khai ứng dụng "Music Player" bằng ngôn ngữ Dart và framework Flutter, nhóm 06 đã hoàn thành các mục tiêu đề ra ban đầu. Dự án không chỉ dừng lại ở một sản phẩm phần mềm mà còn là kết quả của việc áp dụng quy trình phân tích thiết kế hệ thống bài bản.

5.1.1 Những kết quả đạt được

- **Về mặt kỹ thuật:**
 - Xây dựng thành công ứng dụng chạy trên nền tảng di động với cấu trúc mã nguồn rõ ràng, áp dụng mô hình phân lớp (Repository, Manager, Model).
 - Tích hợp thành công các thư viện quan trọng như **just_audio** để xử lý âm thanh và **flutter_localizations** cho đa ngôn ngữ.
 - Đảm bảo tính logic trong việc xử lý danh sách phát, tìm kiếm bài hát và lọc dữ liệu theo Album/Playlist.
- **Về mặt trải nghiệm người dùng (UX):**
 - Giao diện hiện đại, trực quan và có tính tương tác cao (hiệu ứng đĩa nhạc xoay, thanh trượt thời gian thực).
 - Tốc độ phản hồi của ứng dụng nhanh, mượt mà trên cả trình giả lập và thiết bị thật.
- **Về mặt kiến thức:**
 - Nhóm đã nắm vững quy trình xây dựng tài liệu từ đặc tả Use Case, vẽ sơ đồ Activity, Sequence đến thiết kế Class Diagram.

5.1.2 Hạn chế của đề tài

Dù đã đạt được nhiều kết quả khả quan, tuy nhiên do giới hạn về thời gian và nguồn lực, ứng dụng vẫn còn một số điểm cần cải thiện:

- Dữ liệu âm nhạc hiện tại vẫn là dữ liệu nội bộ (Local Data), chưa có sự linh hoạt trong việc cập nhật nội dung mới từ máy chủ.
- Các chức năng bổ sung như tạo Playlist cá nhân hoặc yêu thích bài hát chưa được triển khai sâu.

5.2 Hướng phát triển trong tương lai

Để ứng dụng trở nên hoàn thiện và thực tiễn hơn, nhóm đề xuất các hướng phát triển tiếp theo như sau:

1. **Kết nối Cơ sở dữ liệu trực tuyến:** Tích hợp **Firestore** hoặc **REST API** để lưu trữ và tải nhạc trực tuyến, giúp giảm dung lượng bộ cài đặt ứng dụng.
2. **Quản lý tài khoản người dùng:** Triển khai chức năng Đăng ký/Đăng nhập để đồng bộ hóa danh sách nhạc yêu thích trên nhiều thiết bị.
3. **Tối ưu hóa trải nghiệm âm thanh:** Bổ sung bộ chỉnh âm (Equalizer) và chế độ hẹn giờ tắt nhạc.
4. **Tính năng mạng xã hội:** Cho phép người dùng chia sẻ bài hát đang nghe lên các nền tảng mạng xã hội hoặc bình luận dưới mỗi bài hát.
5. **Phát triển phiên bản Web/Desktop:** Tận dụng tính đa nền tảng của Flutter để đưa ứng dụng lên Windows và nền tảng Web.

TÀI LIỆU THAM KHẢO

1. Tài liệu chuyên ngành và Framework

[1] **Google (2024)**, *Flutter Documentation*, <https://docs.flutter.dev/> – Tài liệu chính thống về Framework Flutter và ngôn ngữ Dart.

[2] **Google (2024)**, *Material Design 3 Guide*, <https://m3.material.io/> – Hướng dẫn về tiêu chuẩn thiết kế giao diện hiện đại cho ứng dụng di động.

[3] **H. Khan (2022)**, *Flutter for Beginners: An introductory guide to building cross-platform mobile applications*, Packt Publishing – Sách hướng dẫn lập trình Flutter cho người mới bắt đầu.

2. Thư viện và Công cụ sử dụng

[4] **Pub.dev**, *just_audio package documentation*, https://pub.dev/packages/just_audio – Tài liệu hướng dẫn sử dụng thư viện xử lý âm thanh trong Flutter.

[5] **Pub.dev**, *flutter_localizations package*, https://api.flutter.dev/flutter/flutter_localizations-library.html – Hướng dẫn triển khai đa ngôn ngữ cho ứng dụng.

[6] **Mermaid.js**, *Diagramming and charting tool*, <https://mermaid.js.org/> – Công cụ hỗ trợ vẽ các sơ đồ hệ thống (Use Case, Activity, Sequence).

3. Giáo trình và Tài liệu học tập

[7] **ThS. Nguyễn Xuân Quế (2024)**, *Bài giảng Lập trình thiết bị di động*, Khoa Công nghệ thông tin – Tài liệu hướng dẫn chuyên môn chính cho học phần.